SCRIPTS:

1. Spectral Analysis (CWT) from raw data traces per cell (run on all 179 embryos)

Purpose:

This pipeline converts the raw fluorescence trace from each cell into a set of quantitative descriptors of its temporal calcium activity. The analysis is performed independently for every cell in all 179 embryos. The pipeline consists of three main stages: preprocessing the fluorescence trace, extracting wavelet-based spectral features, and saving the resulting cell-level measurements.

Driver:

The driver identifies all input files, establishes the frequency range used for spectral analysis, and loops through each embryo and each cell. For every cell, it calls preprocessCellTrace to standardize the temporal signal and extractSpectralFeatures to calculate its wavelet-based spectral characteristics. The resulting measurements are then assembled into a single row of the output table using buildResultRow.
The frequency axis was defined from 0.02 to 0.20 Hz using 80 evenly spaced frequency bins: f_i = linspace(0.02, 0.20, 80)
This defines a common frequency space onto which all cell spectra are interpolated, allowing spectral measurements to be compared across cells even when the CWT returns slightly different frequency sampling.
The driver also saves both the cell-level summary measurements and the complete mean power spectrum for each cell, allowing the derived features to be traced back to the underlying spectral representation.

```
% singlecell_pipelinedriver.m
clear; clc; close all;
%% ======================== CONFIG ========================
inputFolder = 'D:\LAB\SPoriginals';  % <-- set folder containing .xlsx files
filePattern = '*.csv';               % change if you need other extensions
outputRoot = 'D:\LAB\MATLAB_Results';
%% ====================== FIND FILES ======================
files = dir(fullfile(inputFolder, filePattern));
if isempty(files)
   error('No files matching %s found in folder %s', filePattern, inputFolder);
end
%% ====================== INITIALIZE ======================
resultsAll = table();
fMin = 0.02;
fMax = 0.20;
nFreq = 80;
```

```
FrequencyAxis = linspace(fMin, fMax, nFreq);
%% ======================== PIPELINE ========================
for f = 1:numel(files)
   %% Identify file
   inputFile = fullfile(files(f).folder,files(f).name);
   fprintf('\nProcessing file %d/%d: %s\n',...
      f,numel(files),files(f).name);
   %% Create embryo output folders
   [~,baseName,~] = fileparts(files(f).name);
   embryoName = erase(baseName,' - Not_self_Merged');
   embryoFolder = fullfile(outputRoot,embryoName);
   figFolder = fullfile(embryoFolder,'figures');
   tableFolder = fullfile(embryoFolder,'tables');
   if ~isfolder(figFolder)
      mkdir(figFolder)
   end
   if ~isfolder(tableFolder)
      mkdir(tableFolder)
   end
   %% Initialize results for this embryo
   resultsAll = table();
   SpectralLibrary = struct([]);
   %% ===== READ SINGLE SHEET =====
   try
      T = readtable(inputFile);
      SheetName = "CSV";
   catch ME
      warning('Skipping file %s (read failed): %s',...
         files(f).name,ME.message);
      continue
   end
   if isempty(T) || ~istable(T)
      warning('Skipping %s (empty table)',files(f).name);
      continue
   end
   %% ===== FIND CELLS =====
   IDs = unique(T.ID,'stable');
   fprintf('Found %d cells\n',numel(IDs));
   %% ===== CELL LOOP =====
   for c = 1:numel(IDs)
      CellID = IDs(c);
      fprintf('  Processing cell %d/%d (ID=%s)\n',...
         c,numel(IDs),string(CellID));
      %% Extract single cell data
```

```
    idx = T.ID == CellID;
    CellTable = T(idx,:);
    %% ===== STAGE 1: PREPROCESS =====
    try
        data = preprocessCellTrace(CellTable);
    catch ME
        warning('Skipping cell %s preprocessing failed: %s',...
            string(CellID),ME.message);
        continue
    end
    if isfield(data,'skip') && data.skip
        continue
    end
    if ~isfield(data,'X') || ~isfield(data,'Y')
        warning('Skipping cell %s missing spatial data',...
            string(CellID));
        continue
    end
    %% ===== SAVE PREPROCESSED TRACE =====
    fig = figure(...
        'Visible','off',...
        'Position',[100 100 900 300]);
    plot(data.time,data.dFF,'LineWidth',1.5);
    xlabel('Time (s)');
    ylabel('\DeltaF/F');
    title(sprintf('%s | Cell %s | Preprocessed \\DeltaF/F',...
        files(f).name,...
        string(CellID)),...
        'Interpreter','none');
    grid on
    safeBase = regexprep(baseName, '[\x00-\x1F<>:"/\\|?*]', '_');  % remove control + illegal
chars
    safeBase = strtrim(safeBase);                          % remove leading/trailing
whitespace/newlines
    safeID = regexprep(char(string(CellID)), '[\x00-\x1F<>:"/\\|?*]', '_');
    safeID = strtrim(safeID);
    filename = sprintf('%s_Cell_%s.png', safeBase, safeID);
    exportgraphics(fig, fullfile(figFolder, filename), 'Resolution', 300);
    close(fig)
    %% ===== STAGE 2: FEATURE EXTRACTION =====
    try
        [spectralFeat,spectrum] = ...
            extractSpectralFeatures(data,FrequencyAxis);
    catch ME
```

```
            warning('Skipping cell %s feature extraction failed: %s',...
                string(CellID),ME.message);
            continue
        end
        %% ===== BUILD RESULT ROW =====
        newRow = buildResultRow(...
            data,...
            spectralFeat,...
            CellID,...
            files(f).name);
        newRow.SourceFile = string(files(f).name);
        resultsAll = [resultsAll;newRow];
        %% ===== STORE SPECTRAL INFORMATION =====
        k = numel(SpectralLibrary)+1;
        SpectralLibrary(k).CellID = char(string(CellID));
        SpectralLibrary(k).RunID = char(string(CellID));
        SpectralLibrary(k).SheetName = SheetName;
        SpectralLibrary(k).SourceFile = files(f).name;
        SpectralLibrary(k).MeanX = mean(data.X);
        SpectralLibrary(k).MeanY = mean(data.Y);
        SpectralLibrary(k).Frequency = spectrum.Frequency;
        SpectralLibrary(k).Power = spectrum.MeanPower;
    end
    %% ===== SAVE EMBRYO RESULTS =====
    writetable(resultsAll,...
        fullfile(tableFolder,...
        'singlecell_results_final.csv'));
    save(fullfile(embryoFolder,...
        'SpectralLibrary.mat'),...
        'SpectralLibrary',...
        'FrequencyAxis',...
        '-v7.3');
    fprintf('Finished %s\n',embryoName);
end
fprintf('\nAll files complete.\n');
```

Function 1: preprocessCellTrace

Purpose

preprocessCellTrace converts the raw measurements for an individual cell into a clean, uniformly sampled ΔF/F time series suitable for continuous wavelet analysis. It also preserves the cell's spatial coordinates so that spectral features can later be mapped back to cellular position.

Time, fluorescence, and spatial extraction
The function identifies the time column, converts time to seconds, extracts the cell's mean fluorescence intensity, and retrieves its X and Y coordinates. Rows containing missing values in any of these measurements are removed before further processing. This step ensures that the temporal signal, sampling information, and spatial location remain aligned throughout the analysis.
Gap detection
The temporal sampling interval is calculated as: $\Delta t_i = t_{i+1} - t_i$
and the median sampling interval is used as the representative acquisition interval: $\Delta t_{med} = \text{median}(\Delta t_i)$
A temporal gap is defined when: $\Delta t_i > 2\Delta t_{med}$
The resulting indices are used to divide the trace into continuous temporal segments.
The purpose of this segmentation is to prevent a long interruption in imaging from being treated as continuous biological time during interpolation.
Uniform resampling
The sampling frequency is calculated from the median acquisition interval: $Fs = 1/(\Delta t_{med})$
Each continuous segment is independently resampled onto a uniformly spaced time grid using linear interpolation. The original segment endpoints are retained, and only segments containing at least 10 resampled observations are retained for downstream analysis.
Linear interpolation is applied independently to fluorescence and the X/Y coordinates: $F(t) = \text{interp1}(t_i, F_i, t)$
This produces a uniformly sampled signal while avoiding interpolation across the identified acquisition gaps.
Conversion to ΔF/F
The baseline fluorescence is defined as the 10th percentile of the uniformly sampled fluorescence trace: $F_0 = P_{10}(F)$
The baseline is constrained to be positive using machine precision (eps) to prevent division by zero. The normalized fluorescence signal is then calculated as: $F\Delta/F = (F(t) - F_0)/F_0$
Using a percentile-based baseline rather than a single observation provides a robust estimate of the low-fluorescence baseline for traces containing transient calcium elevations.
The resulting ΔF/F trace, sampling frequency, and spatial coordinates are passed to the spectral-analysis function.

```
function data = preprocessCellTrace(T)
data = struct();
data.skip = false;
if ismember('ID',T.Properties.VariableNames)
  uniqueIDs = unique(T.ID);
  if numel(uniqueIDs) > 1
     error('preprocessCellTrace received multiple cell IDs.');
  end
end
%% -------- TIME EXTRACTION --------
vars = T.Properties.VariableNames;
```

```
possibleTimeNames = {'Time','time','Timestamp','timestamp','t','Time_s','Time (s)'};
idx = find(ismember(vars, possibleTimeNames), 1);
if isempty(idx)
  error('No time column found')
end
rawTime = T.(vars{idx});
time = convertTimeToSeconds(rawTime);
%% -------- TRACE EXTRACTION --------
colIdx = find(strcmpi(vars,'MeanIntensity'),1);
if isempty(colIdx)
  colIdx = find(contains(vars,'mean','IgnoreCase',true) & ...
           contains(vars,'intens','IgnoreCase',true),1);
end
trace = double(T{:,colIdx});
% -------- SPATIAL EXTRACTION --------
xIdx = find(strcmpi(vars,'PositionX'),1);
yIdx = find(strcmpi(vars,'PositionY'),1);
if isempty(xIdx) || isempty(yIdx)
  error('No PositionX / PositionY columns found')
end
x = double(T{:,xIdx});
y = double(T{:,yIdx});
valid = ~isnan(trace) & ~isnan(time) & ~isnan(x) & ~isnan(y);
time = time(valid);
trace = trace(valid);
x = x(valid);
y = y(valid);
%% -------- GAP ANALYSIS --------
dt = diff(time);
dt_med = median(dt);
gapIdx = find(dt > 2*dt_med);
data.gaps = gapIdx;
data.dt = dt;
data.dt_med = dt_med;
%% -------- UNIFORM RESAMPLING (KEY FIX) --------
% sanity: time vector must be numeric, non-empty, monotonic
time = double(time(:));
if isempty(time) || numel(time) < 2 || any(~isfinite(time))
  data.skip = true;
  warning('preprocessCellTrace:invalidTime', 'Invalid or too-short time vector.');
  return
end
% compute uniform time vector (already in your code)
dt_med = median(diff(time));
```

```
if ~isfinite(dt_med) || dt_med <= 0
  data.skip = true;
  warning('preprocessCellTrace:badDt', 'Bad median dt (%.4g).', dt_med);
  return
end
% -------- UNIFORM RESAMPLING WITH GAP SEGMENTATION --------
minSamples = 10;    % minimum resampled points required for a usable segment
% recompute/validate time as column vector
time = double(time(:));
if isempty(time) || numel(time) < 2 || any(~isfinite(time))
  error('preprocessCellTrace:InvalidTime', 'Invalid or too-short time vector.');
end
% median dt
dt_med = median(diff(time));
if ~isfinite(dt_med) || dt_med <= 0
  error('preprocessCellTrace:BadDt', 'Bad median dt (%.4g).', dt_med);
end
Fs = 1/dt_med;
% identify gap boundaries (gapIdx already computed earlier in your file)
% gapIdx gives indices i where diff(time)(i) > 2*dt_med, i.e. gap between time(i) and time(i+1)
segmentStarts = [1; gapIdx + 1];
segmentEnds   = [gapIdx; numel(time)];
% containers for concatenated resampled outputs
tUniformAll = [];
traceUniformAll = [];
xUniformAll = [];
yUniformAll = [];
for seg = 1:numel(segmentStarts)
  i1 = segmentStarts(seg);
  i2 = segmentEnds(seg);
  % require at least two source points in the segment
  if (i2 - i1 + 1) < 2
     continue
  end
  t_seg = time(i1:i2);
  trace_seg = trace(i1:i2);
  x_seg = x(i1:i2);
  y_seg = y(i1:i2);
  % unique times within segment (keep first occurrence)
  [t_u, ia] = unique(t_seg, 'stable');
  trace_u = trace_seg(ia);
  x_u = x_seg(ia);
  y_u = y_seg(ia);
  if numel(t_u) < 2
```

```
        continue
    end
    % build resample grid for this segment preserving endpoints
    expectedN = max(2, round((t_u(end) - t_u(1)) / dt_med) + 1);
    tUniform_seg = linspace(t_u(1), t_u(end), expectedN)';
    % require enough resampled points in a segment to be useful
    if numel(tUniform_seg) < minSamples
        continue
    end
    % interpolate (safe: at least 2 source and >=2 target points)
    traceUniform_seg = interp1(t_u, trace_u, tUniform_seg, 'linear', 'extrap');
    xUniform_seg     = interp1(t_u, x_u,     tUniform_seg, 'linear', 'extrap');
    yUniform_seg     = interp1(t_u, y_u,     tUniform_seg, 'linear', 'extrap');
    % append to global arrays
    tUniformAll = [tUniformAll; tUniform_seg]; %#ok<AGROW>
    traceUniformAll = [traceUniformAll; traceUniform_seg]; %#ok<AGROW>
    xUniformAll = [xUniformAll; xUniform_seg]; %#ok<AGROW>
    yUniformAll = [yUniformAll; yUniform_seg]; %#ok<AGROW>
end
% If no valid segments found, fail so caller can log the sheet
if isempty(tUniformAll) || numel(traceUniformAll) < minSamples
    error('preprocessCellTrace:ShortTrace', 'No valid segments with >= %d resampled points
found.', minSamples);
end
% final outputs for downstream processing
tUniform = tUniformAll;
traceUniform = traceUniformAll;
xUniform = xUniformAll;
yUniform = yUniformAll;
% optional: compute and store gap info again for downstream use
data.gaps = gapIdx;
data.dt = diff(tUniform);      % resampled dt vector (may include jumps between segments)
data.dt_med = dt_med;
%% -------- DF/F --------
F0 = prctile(traceUniform,10);
F0 = max(F0, eps);                    % avoid zero or extremely small baseline
dFF = (traceUniform - F0) / F0;
%% OUTPUT
% === CELL METADATA ===
if ismember('ID',T.Properties.VariableNames)
    data.CellID = T.ID(1);
end
% === TIME / SIGNAL OUTPUT ===
data.time = tUniform;
```

```
data.dFF = dFF;
data.Fs = Fs;
% === SPATIAL OUTPUT ===
data.X = xUniform;
data.Y = yUniform;
```

Function 2: extractSpectralFeatures

Purpose

This function characterizes the temporal organization of each cell's calcium activity using the continuous wavelet transform (CWT). Rather than requiring calcium events to be identified individually, the CWT represents how signal power is distributed across temporal frequencies.

The CWT is calculated using MATLAB's analytic Morlet wavelet (amor): [cfs, f_raw] = cwt(dFF, Fs, 'amor')

The resulting complex wavelet coefficients \(W(t,f)\) describe the contribution of each frequency at each time point. Wavelet power is calculated as the squared magnitude of these coefficients: $P(t,f) = |W(t,f)|^2$

The code then averages power across time: $\bar{P}(f) = \frac{1}{T}\sum_{t=1}^{T} P(t,f)$, producing a time-averaged spectral power distribution for each cell.

Frequency-axis standardization

Because the raw CWT frequency vector can differ from the predefined analysis axis, the mean power spectrum is linearly interpolated onto the common 0.02–0.20 Hz, 80-bin frequency axis:

$$\bar{P}_{\text{common}}(f) = \text{interp1}\left(f_{\text{raw}}, \bar{P}_{\text{raw}}, f\right)$$

Values outside the interpolated range are assigned zero.

This creates a common spectral representation for all cells.

Dominant frequency

The dominant frequency is the frequency containing the greatest mean wavelet power:

$$f_{\text{dom}} = \arg\max_{f} \bar{P}(f)$$

In the code, the frequency corresponding to the maximum value of meanPower is assigned as DominantFreqHz.

This provides a measure of the characteristic temporal frequency of calcium activity.

Spectral centroid

The spectral centroid represents the power-weighted mean frequency: $f_c = \frac{\sum_f f\,\bar{P}(f)}{\sum_f \bar{P}(f)}$

Higher values indicate that the cell's spectral power is distributed toward higher frequencies, whereas lower values indicate greater concentration of power at lower frequencies.

Spectral spread

This measures the dispersion of power around the spectral centroid: $\sigma_f = \sqrt{\frac{\sum_f (f - f_c)^2 \bar{P}(f)}{\sum_f \bar{P}(f)}}$

A small value indicates that power is concentrated within a relatively narrow frequency range, whereas a larger value indicates a broader distribution of frequencies.

Spectral entropy

Before calculating entropy, the spectrum is normalized into a probability distribution and then spectral entropy is calculated using Shannon entropy: $P_f = \frac{\bar{P}(f)}{\sum_f \bar{P}(f)}$ $H = -\sum_f P_f \log_2(P_f)$

A lower entropy corresponds to power being concentrated among fewer frequencies, whereas a higher entropy indicates a more distributed spectral organization.

Total band power

This is calculated as the sum of mean power across the predefined frequency range:

$P_{\text{total}} = \sum_f \bar{P}(f)$

This provides a measure of the overall magnitude of oscillatory activity within the analyzed 0.02–0.20 Hz band.

Peak power fraction

Finally, the proportion of total spectral power represented by the dominant frequency is calculated: $R_{peak} = P_{peak} / P_{total}$

This distinguishes cells in which power is strongly concentrated around one dominant frequency from cells whose power is distributed more broadly across the analyzed frequency range.

```
function [spectralFeat, spectrum] = extractSpectralFeatures(data, FrequencyAxis)
dFF = data.dFF;
Fs  = data.Fs;
%% Initialize outputs
spectralFeat = struct( ...
  'DominantFreqHz', NaN, ...
  'SpectralCentroid',    NaN, ...
  'SpectralSpread',      NaN, ...
  'SpectralEntropy',     NaN, ...
  'TotalBandPower',      NaN, ...
  'PeakPowerFraction',   NaN);
spectrum = struct( ...
  'Frequency', [], ...
  'MeanPower', []);
if numel(dFF) < 10
  return
end
%% ==================== CWT ====================
[cfs, f_raw] = cwt(dFF, Fs, 'amor');
```

```
power = abs(cfs).^2;
meanPower_raw = mean(power,2);
%% ==================== ALIGN TO GLOBAL AXIS ====================
meanPower = interp1(f_raw, meanPower_raw, FrequencyAxis, 'linear', 0);
meanPower = max(meanPower, 0);
%% ========== SPECTRAL FEATURES (CONSISTENT FREQUENCY SPACE)
==========
totalPower = sum(meanPower);
if totalPower > 0
  % Normalize spectrum
  P = meanPower / (totalPower + eps);
  % Dominant (maximum-power) frequency
  [peakPower, idx] = max(meanPower);
  spectralFeat.DominantFreqHz = FrequencyAxis(idx);
  % Spectral centroid
  spectralFeat.SpectralCentroid = ...
    sum(FrequencyAxis .* meanPower) / (totalPower + eps);
  % Spectral spread (bandwidth)
  spectralFeat.SpectralSpread = sqrt( ...
    sum(((FrequencyAxis - spectralFeat.SpectralCentroid).^2) .* meanPower) ...
    /(totalPower + eps));
  % Spectral entropy
  spectralFeat.SpectralEntropy = ...
    -sum(P .* log2(P + eps));
  % Total spectral power
  spectralFeat.TotalBandPower = totalPower;
  % Fraction of power contained in dominant frequency
  spectralFeat.PeakPowerFraction = ...
    peakPower / (totalPower + eps);
end
%% ==================== OUTPUT ====================
spectrum.Frequency = FrequencyAxis;
spectrum.MeanPower = meanPower;
end
```

Function 3: buildResultRow

Purpose

buildResultRow combines the spectral measurements with identifying and spatial information into a single cell-level record. Each row contains the cell identifier, source embryo, six spectral measurements, recording duration, and the cell's mean X/Y position.

The mean spatial coordinates are calculated as: $$\bar{X} = \frac{1}{N}\sum_i X_i, \qquad \bar{Y} = \frac{1}{N}\sum_i Y_i$$
These coordinates allow the temporal spectral measurements to subsequently be associated with spatial location.

```
function row = buildResultRow(data, spectralFeat, CellID, sourceFile)
RunID = datetime('now','Format','yyyy-MM-dd_HHmmss');
row = table( ...
  string(CellID), ...
  RunID, ...
  string(sourceFile), ...
  spectralFeat.DominantFreqHz, ...
  spectralFeat.SpectralCentroid, ...
  spectralFeat.SpectralSpread, ...
  spectralFeat.SpectralEntropy, ...
  spectralFeat.TotalBandPower, ...
  spectralFeat.PeakPowerFraction, ...
  (data.time(end)-data.time(1))/60, ...
  mean(data.X), ...
  mean(data.Y), ...
  'VariableNames', { ...
  'CellID',...
  'RunID',...
  'SourceFile',...
  'DominantFreqHz',...
  'SpectralCentroid',...
  'SpectralSpread',...
  'SpectralEntropy',...
  'TotalBandPower',...
  'PeakPowerFraction',...
  'Duration_min',...
  'MeanX',...
  'MeanY'} );
end
```

1. Batch Correlation Script:

Purpose
The batch correlation script tests whether different descriptions of calcium activity covary at the cell level within each embryo and PDT condition. It examines three categories of relationships:
event-detection feature × event-detection feature,
event-detection feature × spectral feature, and
spectral feature × spectral feature.
The four developmental groups are analyzed separately: Stage 13, Stage 13 devitellinated, Stage 14, and Stage 14 devitellinated.

Embryo grouping
The script explicitly assigns each embryo to one of the four experimental groups and checks that no embryo appears in more than one group.
Matching event and spectral measurements
For each embryo, the event-detection table and spectral table are matched using cell identifiers. An innerjoin retains only cells for which both event-based and spectral measurements are available.
Pearson correlation
For every PDT and feature pair, Pearson's correlation coefficient is calculated:

$$r = \frac{\sum_{i=1}^{N}(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^{N}(x_i - \bar{x})^2 \sum_{i=1}^{N}(y_i - \bar{y})^2}}$$

The corresponding \(p\)-value is also calculated using MATLAB's Pearson correlation implementation. A correlation is only calculated when there are at least three valid observations and both variables contain more than one unique value. The analysis asks whether cells with greater values of one calcium-activity feature also tend to have greater or lesser values of another feature within the same embryo/PDT combination. The resulting correlation coefficient, p-value, and number of observations are retained along with the embryo, developmental group, PDT, and identities of the two features.

```
%% ============================================================
% BATCH FEATURE CORRELATION ANALYSIS
% Purpose:
%   Calculate Pearson correlations between:
%       1. Event-detection features
%       2. Spectral/wavelet features
%   for every PDT condition and every embryo.
%
% Groups:
%   Stage 13
%   Stage 13 devitelined
%   Stage 14
```

```
%   Stage 14 devitelined
%
% Summary:
%   Raw PDT-level correlations are saved.
%   These results can later be summarized across embryos/PDTs using median Pearson R and
min-max range.
%% ============================================================
% USER SETTINGS
% ============================================================
% EVENT-DETECTION FILE
% ------------------------------------------------------------
eventFile = ...

'D:\LAB\SPoriginals\Stage_14_devitelined_54_Conditions_Wave_EW_fpf_Unique_LastFrameID
s_Perhour_Counts.csv';
% ------------------------------------------------------------
% ROOT SPECTRAL RESULTS FOLDER
% ------------------------------------------------------------
spectralRoot = ...
   'E:\LAB\MATLAB_Results';
% ------------------------------------------------------------
% OUTPUT FOLDER
% ------------------------------------------------------------
outputFolder = ...
   'E:\LAB\MATLAB_Results\Correlation_Analysis_All_Embryos';
if ~exist(outputFolder,'dir')
   mkdir(outputFolder)
end
%% ============================================================
% EMBRYO GROUP DEFINITIONS
% ============================================================
Stage_13 = {
   '180118SP_E7'
   '180118SP_E8'
   '180118SP_E9'
   '180125SP_E2'
   '180125SP_E3'
   '180216SP_E1'
   '180304SP_E4'
   '180304SP_E5'
   '180315SP_E1'
   '180315SP_E2'
   '180328SP_E6'
   '180328SP_E7'
```

```
    '180405SP_E3'
    '180405SP_E4'
    '180413SP_E3'
    '180427SP_E4'
    '180607SP_E5'
    '180612SP_E2'
    '180612SP_E3'
    '180622SP_E1'
    '180622SP_E6'
    '180628SP_E1'
    '180628SP_E4'
    '180703SP_E2'
    '180703SP_E5'
    '180706SP_E5'
    '180706SP_E6'
    '180802SP_E1'
    '180905SP_E2'
    '180905SP_E5'
    '180905SP_E8'
    '180911SP_E4'
    '190213SP_E2'
    '190918SP_E3'
    '191020SP_E1'
    '191020SP_E2'
    '191020SP_E5'
    '191030SP_E2'
    '191031SP_E1'
    '200114SP_E1'
    '200114SP_E2'
    '200115SP_E1'
    '200217SP_E3'
    '201007SP_E3'
    '201011SP_E1'
    '201023SP_E1'
};
Stage_13_dev = {
    '180509SP_E7'
    '180731SP_E1'
    '180731SP_E2'
    '180731SP_E3'
    '180731SP_E4'
    '180928SP_E7'
    '181002SP_E7'
    '181002SP_E8'
```

```
    '181002SP_E9'
    '181005SP_E2'
    '190208SP_E1'
    '190411SP_E1'
    '190411SP_E2'
    '190411SP_E3'
    '190705SP_E4'
    '190716SP_E1'
    '190717SP_E1'
    '190717SP_E3'
    '190724SP_E2'
    '190724SP_E3'
    '190729SP_E1'
    '190730SP_E7'
    '190831SP_E4'
    '191030SPRN_E3'
    '191103SP_E1'
    '200202SP_E2'
    '200217SP_E2'
    '200301SP_E1'
    '200930SP_E1'
    '200930SP_E5'
    '201001SP_E6'
    '201003SP_E1'
    '201003SP_E2'
    '201003SP_E3'
    '201003SP_E4'
    '201003SP_E7'
    '201004SP_E5'
    '201007SP_E1'
    '201007SP_E2'
    '201007SP_E6'
    '201014SP_E6'
    '201019SP_E2'
    '201019SP_E3'
    '201019SP_E6'
    '201020SP_E2'
    '201023SP_E4'
};
Stage_14 = {
    '180209SP_E4'
    '180209SP_E5'
    '180304SP_E7'
    '180315SP_E3'
```

```
    '180324SP_E1'
    '180328SP_E3'
    '180509SP_E6'
    '180622SP_E2'
    '180628SP_E5'
    '180703SP_E6'
    '180703SP_E7'
    '180711SP_E1'
    '180711SP_E5'
    '180802SP_E2'
    '180802SP_E6'
    '180905SP_E3'
    '180905SP_E4'
    '190206SP_E1'
    '190213SP_E5'
    '190320SP_E1'
    '190320SP_E6'
    '190730SP_E2'
    '190918SP_E4'
    '190918SP_E6'
    '191020RNSP_E6'
    '191020SP_E3'
    '191103SP_E2'
    '191104RN_E2'
    '200114SP_E3'
    '200115SP_E2'
    '200302RN_E5'
    '200313SP_E1'
    '200313SP_E2'
    '201004SP_E4'
    '201007SP_E5'
    '201007SP_E7'
    '201011SP_E5'
    '201014SP_E2'
    '201014SP_E4'
    '201015RN_E3'
    '201015SP_E4'
    '201019SP_E8'
    '201019SP_E9'
    '201020SP_E4'
    '201023SP_E2'
    '201023SP_E5'
};
Stage_14_dev = {
```

```
    '180125SP_E4'
    '180802SP_E5'
    '180921SP_E1'
    '181002SP_E3'
    '181002SP_E4'
    '190327SP_E1'
    '190327SP_E6'
    '190327SP_E7'
    '190401SP_E3'
    '190619SP_E2'
    '190724SP_E5'
    '190724SP_E6'
    '190730SP_E3'
    '190730SP_E4'
    '190730SP_E5'
    '190730SP_E6'
    '190730SP_E8'
    '190918RN_E1'
    '190918SP_E5'
    '200202SP_E4'
    '200202SP_E5'
    '200202SP_E6'
    '200217SP_E5'
    '200301SP_E2'
    '200930SP_E2'
    '200930SP_E3'
    '200930SP_E8'
    '201001SP_E7'
    '201003SP_E5'
    '201003SP_E6'
    '201007SP_E4'
    '201007SP_E8'
    '201011SP_E2'
    '201014SP_E1'
    '201014SP_E7'
    '201015SP_E5'
    '201019SP_E1'
    '201019SP_E4'
    '201020SP_E1'
    '201020SP_E3'
    '201024SP_E1'
};
%% ========================================================
% CONVERT TO STRING ARRAYS
```

```
% ============================================================
Stage_13     = string(Stage_13);
Stage_13_dev = string(Stage_13_dev);
Stage_14     = string(Stage_14);
Stage_14_dev = string(Stage_14_dev);
%% ============================================================
% CREATE GROUP LOOKUP TABLE
% ============================================================
GroupNames = [
   repmat("Stage_13", ...
   numel(Stage_13),1)
   repmat("Stage_13_dev", ...
   numel(Stage_13_dev),1)
   repmat("Stage_14", ...
   numel(Stage_14),1)
   repmat("Stage_14_dev", ...
   numel(Stage_14_dev),1)
   ];
EmbryoNames = [
   Stage_13(:)
   Stage_13_dev(:)
   Stage_14(:)
   Stage_14_dev(:)
   ];
EmbryoGroups = table( ...
   EmbryoNames, ...
   GroupNames, ...
   'VariableNames', ...
   {'Embryo','Group'});
%% ============================================================
% VERIFY GROUP COUNTS
% ============================================================
fprintf('\n============================================================\n')
fprintf('DEFINED EMBRYO GROUPS\n')
fprintf('============================================================\n\n')
fprintf('Stage 13:       %d embryos\n', ...
   numel(Stage_13))
fprintf('Stage 13 dev:   %d embryos\n', ...
   numel(Stage_13_dev))
fprintf('Stage 14:       %d embryos\n', ...
   numel(Stage_14))
fprintf('Stage 14 dev:   %d embryos\n', ...
   numel(Stage_14_dev))
fprintf('--------------------------------\n')
```

```
fprintf('TOTAL:          %d embryos\n\n', ...
  height(EmbryoGroups))
%% ============================================================
% CHECK FOR DUPLICATE EMBRYO ASSIGNMENTS
% ============================================================
[uniqueEmbryos,~,duplicateIdx] = ...
  unique(EmbryoNames);
duplicateCounts = accumarray( ...
  duplicateIdx,1);
duplicateEmbryos = ...
  uniqueEmbryos(duplicateCounts > 1);
if ~isempty(duplicateEmbryos)
  error([ ...
    'The following embryos appear in more than one group:\n', ...
    strjoin(duplicateEmbryos,newline)])
end
%% ============================================================
% READ EVENT DATA
% ============================================================
fprintf('\n============================================================\n')
fprintf('READING EVENT DATA\n')
fprintf('============================================================\n\n')
EventRaw = readtable(eventFile);
fprintf('Event-detection rows: %d\n', ...
  height(EventRaw));
%% ============================================================
% STANDARDIZE EVENT IDENTIFIERS
% ============================================================
if ~ismember('Name', ...
    EventRaw.Properties.VariableNames)
  error('Event file does not contain variable "Name".')
end
if ~ismember('ID', ...
    EventRaw.Properties.VariableNames)
  error('Event file does not contain variable "ID".')
end
EventRaw.Name = string(EventRaw.Name);
EventRaw.CellID = string(EventRaw.ID);
%% ============================================================
% DEFINE EVENT FEATURES
% ============================================================
eventPrefixes = {
  'ActivePeaksPerHr_'
  'Total_'
```

```
    'CellWiseActiveIncidence_'
    'OnlySpikes_'
    'OnlyRosettes_'
    'OnlyWaves_'
};
%% ============================================================
% DEFINE SPECTRAL FEATURES
% ============================================================
spectralFeatures = {
    'DominantFreqHz'
    'SpectralCentroid'
    'SpectralSpread'
    'SpectralEntropy'
    'TotalBandPower'
    'PeakPowerFraction'
};
%% ============================================================
% DETERMINE PDT CONDITIONS
% ============================================================
allVars = ...
    EventRaw.Properties.VariableNames;
activePeakVars = ...
    allVars(startsWith( ...
    allVars, ...
    'ActivePeaksPerHr_'));
if isempty(activePeakVars)
    error( ...
        'No ActivePeaksPerHr_ variables were found.')
end
PDTs = string( ...
    erase( ...
    activePeakVars, ...
    'ActivePeaksPerHr_'));
fprintf('\n============================================================\n')
fprintf('PDT CONDITIONS\n')
fprintf('============================================================\n\n')
fprintf('Number of PDTs: %d\n\n', ...
    numel(PDTs))
disp(PDTs')
%% ============================================================
% FIND SPECTRAL FILES
% ============================================================
fprintf('\n============================================================\n')
fprintf('SEARCHING FOR SPECTRAL FILES\n')
```

```
fprintf('============================================================\n\n')
% Get folders directly inside MATLAB_Results
embryoFolders = dir(spectralRoot);
embryoFolders = ...
  embryoFolders([embryoFolders.isdir]);
embryoFolders = ...
  embryoFolders(~ismember( ...
  {embryoFolders.name}, ...
  {'.','..'}));
fprintf('Embryo folders found: %d\n\n', ...
  numel(embryoFolders));
%% ------------------------------------------------------------
% Build spectral file list
% ------------------------------------------------------------
spectralFiles = struct( ...
  'name',{}, ...
  'folder',{});
for k = 1:numel(embryoFolders)
  embryoFolder = ...
    fullfile( ...
    spectralRoot, ...
    embryoFolders(k).name);
  tablesFolder = ...
    fullfile( ...
    embryoFolder, ...
    'tables');
  if ~isfolder(tablesFolder)
    fprintf('No tables folder: %s\n', ...
      embryoFolders(k).name)
    continue
  end
  csvFile = dir( ...
    fullfile( ...
    tablesFolder, ...
    'singlecell_results_final.csv'));
  if isempty(csvFile)
    fprintf('No spectral CSV: %s\n', ...
      embryoFolders(k).name)
    continue
  end
  spectralFiles(end+1).name = ...
    csvFile(1).name;
  spectralFiles(end).folder = ...
    csvFile(1).folder;
```

```
end
fprintf('\nSpectral CSV files found: %d\n\n', ...
    numel(spectralFiles));
if isempty(spectralFiles)
    error([ ...
        'No singlecell_results_final.csv files were found ', ...
        'inside embryo tables folders.'])
end
%% ============================================================
% INITIALIZE MASTER RESULTS TABLE
% ============================================================
allResults = table( ...
    strings(0,1), ...
    strings(0,1), ...
    strings(0,1), ...
    strings(0,1), ...
    strings(0,1), ...
    strings(0,1), ...
    zeros(0,1), ...
    zeros(0,1), ...
    zeros(0,1), ...
    'VariableNames',{ ...
    'Embryo', ...
    'Group', ...
    'PDT', ...
    'CorrelationType', ...
    'Feature1', ...
    'Feature2', ...
    'N', ...
    'PearsonR', ...
    'PValue'});
%% ============================================================
% PROCESS EACH EMBRYO
% ============================================================
for embryoIndex = 1:height(EmbryoGroups)
    fprintf('\n============================================================\n')
    fprintf('EMBRYO %d / %d\n', ...
        embryoIndex, ...
        height(EmbryoGroups))
    fprintf('============================================================\n')
    embryoCore = ...
        EmbryoGroups.Embryo(embryoIndex);
    groupName = ...
        EmbryoGroups.Group(embryoIndex);
```

```
fprintf('Embryo: %s\n', ...
    embryoCore)
fprintf('Group:  %s\n\n', ...
    groupName)
%% ========================================================
% FIND MATCHING SPECTRAL FILE
% ========================================================
matchingFiles = struct( ...
    'name',{}, ...
    'folder',{});
embryoNormalized = ...
    erase(embryoCore,"_");
for f = 1:numel(spectralFiles)
    % ----------------------------------------------
    % Get embryo folder
    % ----------------------------------------------
    thisTablesFolder = ...
        spectralFiles(f).folder;
    thisEmbryoFolder = ...
        fileparts(thisTablesFolder);
    [~,folderName] = ...
        fileparts(thisEmbryoFolder);
    % ----------------------------------------------
    % Normalize folder name
    % ----------------------------------------------
    folderNormalized = ...
        erase(string(folderName),"_");
    token = regexp( ...
        char(folderNormalized), ...
        '^(.*?E\d+)', ...
        'tokens', ...
        'once');
    if isempty(token)
        continue
    end
    spectralEmbryoCore = ...
        string(token{1});
    % ----------------------------------------------
    % Compare embryo cores
    % ----------------------------------------------
    if strcmpi( ...
            spectralEmbryoCore, ...
            embryoNormalized)
        matchingFiles(end+1) = ...
```

```
            spectralFiles(f);
    end
end
%% ========================================================
% CHECK MATCH
% ========================================================
if isempty(matchingFiles)
    warning( ...
        'No spectral CSV found for embryo %s. Skipping.', ...
        embryoCore)
    continue
end
if numel(matchingFiles) > 1
    warning( ...
        'Multiple spectral CSVs found for embryo %s. Skipping.', ...
        embryoCore)
    for f = 1:numel(matchingFiles)
        fprintf('  %s\n', ...
            matchingFiles(f).folder)
    end
    continue
end
%% ========================================================
% BUILD SPECTRAL FILE PATH
% ========================================================
waveletFile = ...
    fullfile( ...
    matchingFiles.folder, ...
    matchingFiles.name);
fprintf('Spectral file:\n%s\n\n', ...
    waveletFile);
%% ========================================================
% READ SPECTRAL DATA
% ========================================================
try
    Wavelet = ...
        readtable(waveletFile);
catch ME
    warning( ...
        'Could not read spectral file for %s:\n%s', ...
        embryoCore, ...
        ME.message)
    continue
end
```

```
%% ========================================================
% CHECK CELLID
% ========================================================
if ~ismember( ...
        'CellID', ...
        Wavelet.Properties.VariableNames)
    warning( ...
        'CellID missing from spectral file for %s. Skipping.', ...
        embryoCore)
    continue
end
Wavelet.CellID = ...
    string(Wavelet.CellID);
%% ========================================================
% MATCH EVENT EMBRYO
% ========================================================
eventEmbryos = ...
    unique(EventRaw.Name);
eventEmbryosNormalized = ...
    erase(eventEmbryos,"_");
targetEmbryoNormalized = ...
    erase(embryoCore,"_");
matchIdx = strcmpi( ...
    eventEmbryosNormalized, ...
    targetEmbryoNormalized);
if sum(matchIdx) == 0
    warning( ...
        'No event-data match found for embryo %s. Skipping.', ...
        embryoCore)
    continue
elseif sum(matchIdx) > 1
    warning( ...
        'Multiple event-data matches found for embryo %s. Skipping.', ...
        embryoCore)
    disp(eventEmbryos(matchIdx))
    continue
end
embryoName = ...
    eventEmbryos(find(matchIdx,1));
fprintf('Event embryo name: %s\n', ...
    embryoName)
%% ========================================================
% FILTER EVENT DATA
% ========================================================
```

```
EventEmbryo = ...
    EventRaw( ...
    EventRaw.Name == embryoName,:);
%% ========================================================
% MERGE EVENT + SPECTRAL DATA
% ========================================================
Tall = ...
    innerjoin( ...
    EventEmbryo, ...
    Wavelet, ...
    'Keys','CellID');
fprintf('Event cells:   %d\n', ...
    height(EventEmbryo))
fprintf('Spectral cells: %d\n', ...
    height(Wavelet))
fprintf('Merged cells:  %d\n\n', ...
    height(Tall))
if height(Tall) < 3
    warning( ...
        'Too few merged cells for %s. Skipping.', ...
        embryoCore)
    continue
end
%% ========================================================
% CHECK SPECTRAL VARIABLES
% ========================================================
missingSpectral = ...
    setdiff( ...
    spectralFeatures, ...
    Tall.Properties.VariableNames);
if ~isempty(missingSpectral)
    warning( ...
        'Missing spectral variables for %s: %s. Skipping.', ...
        embryoCore, ...
        strjoin(missingSpectral,", "))
    continue
end
%% ========================================================
% CALCULATE CORRELATIONS
% ========================================================
fprintf('Calculating correlations...\n')
for p = 1:numel(PDTs)
    thisPDT = PDTs(p);
    %% ====================================================
```

```
% EVENT-EVENT + EVENT-SPECTRAL
% ================================================
for e = 1:numel(eventPrefixes)
    eventVar = ...
        [eventPrefixes{e},char(thisPDT)];
    if ~ismember( ...
            eventVar, ...
            Tall.Properties.VariableNames)
        continue
    end
    %% ---------------------------------------------
    % EVENT-EVENT
    % ---------------------------------------------
    for e2 = 1:numel(eventPrefixes)
        eventVar2 = ...
            [eventPrefixes{e2},char(thisPDT)];
        if ~ismember( ...
                eventVar2, ...
                Tall.Properties.VariableNames)
            continue
        end
        x = Tall{:,eventVar};
        y = Tall{:,eventVar2};
        if ~isnumeric(x)
            x = str2double(string(x));
        end
        if ~isnumeric(y)
            y = str2double(string(y));
        end
        valid = ...
            isfinite(x) & ...
            isfinite(y);
        N = sum(valid);
        if N >= 3 && ...
                numel(unique(x(valid))) > 1 && ...
                numel(unique(y(valid))) > 1
            [r,pval] = ...
                corr( ...
                x(valid), ...
                y(valid), ...
                'Type','Pearson');
        else
            r = NaN;
            pval = NaN;
```

```
    end
    %% ---------------------------------------------
    % STORE EVENT-EVENT RESULT
    % ---------------------------------------------
    newRow = table( ...
        string(embryoCore), ...
        string(groupName), ...
        string(thisPDT), ...
        "Event-Event", ...
        string(eventPrefixes{e}), ...
        string(eventPrefixes{e2}), ...
        N, ...
        r, ...
        pval, ...
        'VariableNames',{ ...
        'Embryo', ...
        'Group', ...
        'PDT', ...
        'CorrelationType', ...
        'Feature1', ...
        'Feature2', ...
        'N', ...
        'PearsonR', ...
        'PValue'});
    allResults = ...
        [allResults;newRow];
end
%% ---------------------------------------------
% EVENT-SPECTRAL
% ---------------------------------------------
for s = 1:numel(spectralFeatures)
    spectralVar = ...
        spectralFeatures{s};
    x = Tall{:,eventVar};
    y = Tall{:,spectralVar};
    if ~isnumeric(x)
        x = str2double(string(x));
    end
    if ~isnumeric(y)
        y = str2double(string(y));
    end
    valid = ...
        isfinite(x) & ...
        isfinite(y);
```

```
        N = sum(valid);
        if N >= 3 && ...
                numel(unique(x(valid))) > 1 && ...
                numel(unique(y(valid))) > 1
            [r,pval] = ...
                corr( ...
                x(valid), ...
                y(valid), ...
                'Type','Pearson');
        else
            r = NaN;
            pval = NaN;
        end
        %% ---------------------------------------------
        % STORE EVENT-SPECTRAL RESULT
        % ---------------------------------------------
        newRow = table( ...
            string(embryoCore), ...
            string(groupName), ...
            string(thisPDT), ...
            "Event-Spectral", ...
            string(eventPrefixes{e}), ...
            string(spectralFeatures{s}), ...
            N, ...
            r, ...
            pval, ...
            'VariableNames',{ ...
            'Embryo', ...
            'Group', ...
            'PDT', ...
            'CorrelationType', ...
            'Feature1', ...
            'Feature2', ...
            'N', ...
            'PearsonR', ...
            'PValue'});
        allResults = ...
            [allResults;newRow];
    end
end
%% ================================================
% SPECTRAL-SPECTRAL
% ================================================
for s1 = 1:numel(spectralFeatures)
```

```
for s2 = 1:numel(spectralFeatures)
    x = ...
        Tall{:,spectralFeatures{s1}};
    y = ...
        Tall{:,spectralFeatures{s2}};
    if ~isnumeric(x)
        x = str2double(string(x));
    end
    if ~isnumeric(y)
        y = str2double(string(y));
    end
    valid = ...
        isfinite(x) & ...
        isfinite(y);
    N = sum(valid);
    if N >= 3 && ...
            numel(unique(x(valid))) > 1 && ...
            numel(unique(y(valid))) > 1
        [r,pval] = ...
            corr( ...
            x(valid), ...
            y(valid), ...
            'Type','Pearson');
    else
        r = NaN;
        pval = NaN;
    end
    %% ---------------------------------------------
    % STORE SPECTRAL-SPECTRAL RESULT
    % ---------------------------------------------
    newRow = table( ...
        string(embryoCore), ...
        string(groupName), ...
        string(thisPDT), ...
        "Spectral-Spectral", ...
        string(spectralFeatures{s1}), ...
        string(spectralFeatures{s2}), ...
        N, ...
        r, ...
        pval, ...
        'VariableNames',{ ...
        'Embryo', ...
        'Group', ...
        'PDT', ...
```

```
                'CorrelationType', ...
                'Feature1', ...
                'Feature2', ...
                'N', ...
                'PearsonR', ...
                'PValue'});
            allResults = ...
                [allResults;newRow];
        end
    end
  end
  %% ============================================================
  % EMBRYO COMPLETE
  % ============================================================
  fprintf('\nFinished %s\n', ...
      embryoCore)
  fprintf('Total correlation rows accumulated: %d\n', ...
      height(allResults))
end
%% ================================================================
% SAVE RAW RESULTS
% ================================================================
fprintf('\n================================================================\n')
fprintf('SAVING RESULTS\n')
fprintf('================================================================\n\n')
if isempty(allResults)
  warning( ...
      'allResults is empty. No correlations were successfully generated.')
else
  %% ------------------------------------------------------
  % CSV
  % ------------------------------------------------------
  csvOutput = ...
      fullfile( ...
      outputFolder, ...
      '14dev_All_Embryos_All_PDT_Correlations.csv');
  writetable( ...
      allResults, ...
      csvOutput);
  fprintf('CSV saved:\n%s\n\n', ...
      csvOutput);
end
%% ================================================================
% FINAL SUMMARY
```

```
% ============================================================
fprintf('\n============================================================\n')
fprintf('ANALYSIS COMPLETE\n')
fprintf('============================================================\n\n')
fprintf('Embryos in requested groups: %d\n', ...
  height(EmbryoGroups))
fprintf('Embryos represented in results: %d\n', ...
  numel(unique(allResults.Embryo)))
fprintf('PDTs analyzed: %d\n', ...
  numel(PDTs))
fprintf('Total correlation rows: %d\n', ...
  height(allResults))
fprintf('\nCorrelation types:\n')
if ~isempty(allResults)
  fprintf('  Event-Event:       %d\n', ...
    sum(allResults.CorrelationType == "Event-Event"))
  fprintf('  Event-Spectral:    %d\n', ...
    sum(allResults.CorrelationType == "Event-Spectral"))
  fprintf('  Spectral-Spectral: %d\n', ...
    sum(allResults.CorrelationType == "Spectral-Spectral"))
end
fprintf('\nOutput folder:\n%s\n', ...
  outputFolder)
fprintf('\n============================================================\n')
```

2. Batch Correlation Plots:

Purpose

The plotting script summarizes the individual embryo/PDT correlations generated by the previous analysis. For each developmental group, it produces a 12 × 12 feature matrix containing six event-detection features and six spectral features. For each feature pair, the script collects all valid Pearson r values across embryos and PDT conditions and calculates:

- median Pearson r,
- minimum Pearson r,
- maximum Pearson r,
- standard deviation of Pearson r, and
- number of correlations contributing to the summary.

The median is therefore calculated across embryo/PDT-level correlation coefficients, rather than by pooling all cells from all embryos into a single correlation.

The resulting matrix is displayed as a heatmap with a fixed color range of: $-1 \leq r \leq 1$

Each cell of the heatmap is annotated with the median correlation and the observed minimum-to-maximum range: median r [min r, max r]

This allows the central tendency of the relationship and its variability across embryos/PDT conditions to be viewed simultaneously.

```
%% ============================================================
% GROUP-LEVEL FEATURE CORRELATION PLOTTING
%
% Purpose:
%   Plot the correlation results already calculated separately
%   for each developmental group.
%
% For each group:
%   - 6 event-detection features
%   - 6 spectral features
%   - Event × Event correlations
%   - Event × Spectral correlations
%   - Spectral × Spectral correlations
%
% Summary:
%   Cell color = MEDIAN Pearson r
%   Cell annotation = [MIN, MAX] range
%
% IMPORTANT:
%   Each developmental group is kept completely separate because
%   each group was analyzed using its own event-detection file.
%
% Final matrix:
%   12 × 12
% Feature order:
%   1-6  = Event features
%   7-12 = Spectral features
%% ============================================================
% USER SETTINGS
% ============================================================
% ROOT FOLDER CONTAINING THE FOUR GROUP RESULTS
% ------------------------------------------------------------
resultsRoot = ...
   'E:\LAB\MATLAB_Results\Correlation_Analysis_All_Embryos';
% ------------------------------------------------------------
% OUTPUT FOLDER FOR FIGURES
% ------------------------------------------------------------
outputFolder = ...
   'E:\LAB\MATLAB_Results\Correlation_Analysis_All_Embryos\Figures';
if ~exist(outputFolder,'dir')
```

```
    mkdir(outputFolder)
end
%% ============================================================
% DEFINE GROUPS
% ============================================================
groupNames = {
    'Stage_13'
    'Stage_13_dev'
    'Stage_14'
    'Stage_14_dev'
    };
%% ============================================================
% DEFINE EVENT-DETECTION FEATURES
% ============================================================
eventFeatures = {
    'ActivePeaksPerHr_'
    'Total_'
    'CellWiseActiveIncidence_'
    'OnlySpikes_'
    'OnlyRosettes_'
    'OnlyWaves_'
    };
%% ============================================================
% DEFINE SPECTRAL FEATURES
% ============================================================
spectralFeatures = {
    'DominantFreqHz'
    'SpectralCentroid'
    'SpectralSpread'
    'SpectralEntropy'
    'TotalBandPower'
    'PeakPowerFraction'
    };
%% ============================================================
% COMBINED FEATURE LIST
% ============================================================
allFeatureNames = [
    eventFeatures(:)
    spectralFeatures(:)
    ];
nEvent = numel(eventFeatures);
nSpectral = numel(spectralFeatures);
nFeatures = numel(allFeatureNames);
fprintf('\n============================================================\n')
```

```
fprintf('GROUP CORRELATION PLOTTING\n')
fprintf('============================================================\n')
fprintf('\nFeatures: %d\n',nFeatures)
fprintf('Event features: %d\n',nEvent)
fprintf('Spectral features: %d\n',nSpectral)
%% ============================================================
% PROCESS EACH GROUP
% ============================================================
for g = 1:numel(groupNames)
  groupName = groupNames{g};
  fprintf('\n============================================================\n')
  fprintf('GROUP %d OF %d: %s\n', ...
     g,numel(groupNames),groupName)
  fprintf('============================================================\n')
  %% ============================================================
  % FIND RESULTS FILE
  % ============================================================
  % All four result files are stored directly in resultsRoot.
  % The group is identified by the prefix in the filename.

  switch groupName

     case 'Stage_13'
        filePrefix = '13';

     case 'Stage_13_dev'
        filePrefix = '13dev';

     case 'Stage_14'
        filePrefix = '14';

     case 'Stage_14_dev'
        filePrefix = '14dev';

     otherwise
        warning( ...
           'Unknown group name: %s. Skipping.', ...
           groupName);
        continue

  end


  resultsFile = fullfile( ...
```

```
    resultsRoot, ...
    [filePrefix, '_All_Embryos_All_PDT_Correlations.csv']);


%% ------------------------------------------------------
% Check that file exists
% ------------------------------------------------------

if ~isfile(resultsFile)

    warning( ...
        ['Results file not found for %s.\n', ...
         'Expected:\n%s\n', ...
         'Skipping group.'], ...
        groupName, ...
        resultsFile);

    continue

end


fprintf('\nReading results:\n%s\n',resultsFile)
%% ======================================================
% READ RESULTS
% ======================================================
results = readtable(resultsFile);
fprintf('Rows loaded: %d\n',height(results))
%% ======================================================
% CHECK REQUIRED VARIABLES
% ======================================================
requiredVariables = {
    'Embryo'
    'PDT'
    'Feature1'
    'Feature2'
    'PearsonR'
    };
for v = 1:numel(requiredVariables)
    if ~ismember( ...
            requiredVariables{v}, ...
            results.Properties.VariableNames)
        error( ...
            ['Results file for %s is missing required ', ...
```

```
            'variable: %s'], ...
            groupName, ...
            requiredVariables{v});
    end
end
%% ========================================================
% STANDARDIZE IDENTIFIERS
% ========================================================
results.Embryo = string(results.Embryo);
results.PDT = string(results.PDT);
results.Feature1 = string(results.Feature1);
results.Feature2 = string(results.Feature2);
%% ========================================================
% BASIC SUMMARY
% ========================================================
embryos = unique(results.Embryo);
PDTlist = unique(results.PDT);
fprintf('Embryos represented: %d\n', ...
    numel(embryos))
fprintf('PDTs represented: %d\n', ...
    numel(PDTlist))
%% ========================================================
% INITIALIZE SUMMARY MATRICES
% ========================================================
Rmedian = NaN(nFeatures,nFeatures);
Rmin = NaN(nFeatures,nFeatures);
Rmax = NaN(nFeatures,nFeatures);
Rstd = NaN(nFeatures,nFeatures);
Nvalues = zeros(nFeatures,nFeatures);
%% ========================================================
% SUMMARIZE EACH FEATURE PAIR
% ========================================================
for i = 1:nFeatures
    for j = i:nFeatures
        feature1 = string(allFeatureNames{i});
        feature2 = string(allFeatureNames{j});
        %% ------------------------------------------------
        % Find matching rows
        % ------------------------------------------------
        match = ...
            results.Feature1 == feature1 & ...
            results.Feature2 == feature2;
        rValues = results.PearsonR(match);
        %% ------------------------------------------------
```

```
        % Remove invalid correlations
        % ------------------------------------------------
        rValues = rValues(isfinite(rValues));
        %% ------------------------------------------------
        % Calculate summary statistics
        % ------------------------------------------------
        if ~isempty(rValues)
            Rmedian(i,j) = median(rValues);
            Rmin(i,j) = min(rValues);
            Rmax(i,j) = max(rValues);
            Rstd(i,j) = std(rValues);
            Nvalues(i,j) = numel(rValues);
        end
        %% ------------------------------------------------
        % Mirror matrix
        % ------------------------------------------------
        Rmedian(j,i) = Rmedian(i,j);
        Rmin(j,i) = Rmin(i,j);
        Rmax(j,i) = Rmax(i,j);
        Rstd(j,i) = Rstd(i,j);
        Nvalues(j,i) = Nvalues(i,j);
    end
end
%% ========================================================
% SAVE SUMMARY MATRICES
% ========================================================
fprintf('\nSaving summary matrices...\n')
% --------------------------------------------------------
% Median
% --------------------------------------------------------
MedianTable = array2table( ...
    Rmedian, ...
    'VariableNames',allFeatureNames, ...
    'RowNames',allFeatureNames);
writetable( ...
    MedianTable, ...
    fullfile( ...
    outputFolder, ...
    sprintf('%s_Median_R.xlsx',groupName)), ...
    'WriteRowNames',true);
% --------------------------------------------------------
% Minimum
% --------------------------------------------------------
MinTable = array2table( ...
```

```
    Rmin, ...
    'VariableNames',allFeatureNames, ...
    'RowNames',allFeatureNames);
writetable( ...
    MinTable, ...
    fullfile( ...
    outputFolder, ...
    sprintf('%s_Min_R.xlsx',groupName)), ...
    'WriteRowNames',true);
% ------------------------------------------------------
% Maximum
% ------------------------------------------------------
MaxTable = array2table( ...
    Rmax, ...
    'VariableNames',allFeatureNames, ...
    'RowNames',allFeatureNames);
writetable( ...
    MaxTable, ...
    fullfile( ...
    outputFolder, ...
    sprintf('%s_Max_R.xlsx',groupName)), ...
    'WriteRowNames',true);
% ------------------------------------------------------
% Standard deviation
% ------------------------------------------------------
StdTable = array2table( ...
    Rstd, ...
    'VariableNames',allFeatureNames, ...
    'RowNames',allFeatureNames);
writetable( ...
    StdTable, ...
    fullfile( ...
    outputFolder, ...
    sprintf('%s_Std_R.xlsx',groupName)), ...
    'WriteRowNames',true);
% ------------------------------------------------------
% Number of correlations contributing to each cell
% ------------------------------------------------------
NTable = array2table( ...
    Nvalues, ...
    'VariableNames',allFeatureNames, ...
    'RowNames',allFeatureNames);
writetable( ...
    NTable, ...
```

```
    fullfile( ...
    outputFolder, ...
    sprintf('%s_N_Correlations.xlsx',groupName)), ...
    'WriteRowNames',true);
%% ============================================================
% CREATE FIGURE
% ============================================================
fprintf('Creating figure...\n')
fig = figure( ...
    'Visible','off', ...
    'Color','w', ...
    'Position',[100 100 1500 1300]);
imagesc(Rmedian);
axis square
colormap(parula)
colorbar
clim([-1 1])
%% ============================================================
% AXIS LABELS
% ============================================================
xticks(1:nFeatures)
xticklabels(allFeatureNames)
xtickangle(45)
yticks(1:nFeatures)
yticklabels(allFeatureNames)
xlabel('Feature')
ylabel('Feature')
title({ ...
    sprintf('%s',groupName), ...
    'Median Pearson Correlation Across Embryos and PDTs'}, ...
    'Interpreter','none');
%% ============================================================
% ANNOTATE MEDIAN + RANGE
% ============================================================
for i = 1:nFeatures
    for j = 1:nFeatures
        if ~isfinite(Rmedian(i,j))
            continue
        end
        %% ---------------------------------------------
        % Text color
        % ---------------------------------------------
        if abs(Rmedian(i,j)) >= 0.60
            txtColor = 'w';
```

```
        else
            txtColor = 'k';
        end
        %% ---------------------------------------------
        % Median r
        % ---------------------------------------------
        medianText = sprintf( ...
            '%.2f', ...
            Rmedian(i,j));
        %% ---------------------------------------------
        % Range
        % ---------------------------------------------
        rangeText = sprintf( ...
            '[%.2f, %.2f]', ...
            Rmin(i,j), ...
            Rmax(i,j));
        %% ---------------------------------------------
        % Add annotation
        % ---------------------------------------------
        text( ...
            j, ...
            i, ...
            {medianText;rangeText}, ...
            'HorizontalAlignment','center', ...
            'VerticalAlignment','middle', ...
            'Color',txtColor, ...
            'FontSize',7, ...
            'FontWeight','bold');
    end
end
%% ========================================================
% SEPARATE EVENT AND SPECTRAL BLOCKS
% ========================================================
hold on
% Vertical separator
xline( ...
    nEvent + 0.5, ...
    'k-', ...
    'LineWidth',2);
% Horizontal separator
yline( ...
    nEvent + 0.5, ...
    'k-', ...
    'LineWidth',2);
```

```
    hold off
    %% ========================================================
    % ADD GROUP LABELS
    % ========================================================
    % Event label above left block
    text( ...
        nEvent/2, ...
        -0.15, ...
        'EVENT FEATURES', ...
        'Units','data', ...
        'HorizontalAlignment','center', ...
        'FontWeight','bold', ...
        'FontSize',11);
    % Spectral label above right block
    text( ...
        nEvent + nSpectral/2, ...
        -0.15, ...
        'SPECTRAL FEATURES', ...
        'Units','data', ...
        'HorizontalAlignment','center', ...
        'FontWeight','bold', ...
        'FontSize',11);
    %% ========================================================
    % SAVE FIGURE
    % ========================================================
    figureFile = fullfile( ...
        outputFolder, ...
        sprintf('%s_Median_Correlation_Matrix.png', ...
        groupName));
    exportgraphics( ...
        fig, ...
        figureFile, ...
        'Resolution',300);
    close(fig);
    fprintf('\nFigure saved:\n%s\n',figureFile)
end
%% ============================================================
% COMPLETE
% ============================================================
fprintf('\n============================================================\n')
fprintf('ALL GROUP FIGURES COMPLETE\n')
fprintf('============================================================\n')
fprintf('\nOutput folder:\n%s\n',outputFolder)
fprintf('\nGroups processed:\n')
```

```
for g = 1:numel(groupNames)
   fprintf('  %s\n',groupNames{g})
end
fprintf('\n============================================================\n')
```

SCRIPTS:

Spatial coordinate standardization
The spatial-mapping pipeline combines the cell-level spectral results with the corresponding embryo boundaries and assigns each cell to a common 9 × 9 pixel CNPU grid. The configuration explicitly sets:
CNPUsize = 9
GridOriginX = 120
GridOriginY = 1

The purpose is to place cells from different embryos into a common coordinate system so that spatial distributions can be compared across embryos.
The center of each CNPU is calculated from its grid index and the specified grid origin and size. The complete legacy grid is retained as the common spatial canvas rather than restricting the analysis to the Sox2 boundary.

Embryo resolution
resolveMappedEmbryos identifies the mapped source file corresponding to each biological embryo and resolves duplicate source files so that each biological embryo contributes only one mapped dataset. The function explicitly checks that the final number of biological embryos is 179.

Loading and validation
loadMappedEmbryoData loads the mapped cell-level data, verifies embryo/group assignments, checks required identifiers and spatial coordinates, and constructs the common CNPU grid. The function also identifies which CNPUs were observed and which were unobserved.
This distinction is important for interpreting spatial maps: an uncolored region can represent lack of observed cells, rather than a measured low feature value.

Template boundaries
The Sox2 boundaries are used as anatomical overlays, not as masks on the data. The mapping code explicitly preserves the complete CNPU grid and plots the Sox2 boundary on top of it.
For Stage 13 groups, the Stage 14 Sox2 template is used; for Stage 14 groups, the Stage 15 Sox2 template is used. The boundary function reads the appropriate template, extracts X/Y coordinates, removes invalid points, and closes the polygon if necessary.

1. Boundary Check

```
%% ===== centroid + boundary check ===== %%
% figures show embryo map by cell centroids, cell boundaries, & one cell
% confirmed centroid inside mapped boundaries
% Set the full path to the CSV file
filepath = 'E:\LAB\Boundaries\Stage_13\180118SP_E7-Boundary_SuperEmbryo.csv';
T = readtable(filepath);
```

```
figure
scatter(T.Centroid_1, T.Centroid_2, 10, 'filled')
axis equal
xlabel('Template X')
ylabel('Template Y')
title('SuperEmbryo mapped cell centroids')
boundaryNames =
T.Properties.VariableNames(startsWith(T.Properties.VariableNames,'Boundary'));
% make sure Boundary_1, Boundary_2, ..., Boundary_128 are ordered numerically
boundaryNumbers = cellfun(@(x) sscanf(x,'Boundary_%d'), boundaryNames);
[~,idx] = sort(boundaryNumbers);
boundaryNames = boundaryNames(idx);
figure
hold on
for i = 1:10:height(T)
  boundary = table2array(T(i,boundaryNames));
  boundary = boundary(~isnan(boundary));
  if isempty(boundary)
     continue
  end
  n = length(boundary)/2;
  Y = boundary(1:n);
  X = boundary(n+1:end);
  plot(X,Y)
end
axis equal
title('Mapped cell boundaries')
boundaryNames =
T.Properties.VariableNames(startsWith(T.Properties.VariableNames,'Boundary'));
validCount = 0;
for i = 1:height(T)
  boundary = table2array(T(i,boundaryNames));
  boundary = boundary(~isnan(boundary));
  if ~isempty(boundary)
     validCount = validCount + 1;
  end
end
disp(validCount)
cell = 100;
figure
hold on
scatter(T.Centroid_1(cell),T.Centroid_2(cell),100,'filled')
boundary = table2array(T(cell,boundaryNames));
boundary = boundary(~isnan(boundary));
```

```
n=length(boundary)/2;
Y=boundary(1:n);
X=boundary(n+1:end);
plot(X,Y,'LineWidth',2)
axis equal
```

2. Individual mapping:

Individual embryo mapping: produces the [embryo name]_Mapped_[Spectral Feature] PNGs for each embryo found in their MATLAB_Results

```
%% ==== Batch Mapping spectral features to SuperEmbryo template ==== %%
clear; clc; close all
%% Root folders
BoundaryRoot = 'E:\LAB\Boundaries';
ResultsRoot  = 'E:\LAB\MATLAB_Results';
%% Features to plot
featuresToPlot = { ...
  'DominantFreqHz',...
  'SpectralCentroid',...
  'SpectralSpread',...
  'SpectralEntropy',...
  'TotalBandPower',...
  'PeakPowerFraction'};
%% ================================================================
%  FIRST PASS:
%  Load and merge all embryos, then determine global plot limits
%  ================================================================
%% Find all boundary files
BoundaryFiles = dir(fullfile(BoundaryRoot,...
  '*-Boundary_SuperEmbryo.csv'));
fprintf('Found %d boundary files\n\n',length(BoundaryFiles));
%% Storage for merged embryo data
AllMerged = cell(length(BoundaryFiles),1);
EmbryoNames = cell(length(BoundaryFiles),1);
%% Global spatial limits
AllX = [];
AllY = [];
%% Global feature values
AllFeatureValues = struct();
for n = 1:length(BoundaryFiles)
  fprintf('========================================\n');
  fprintf('Loading embryo %d of %d\n',n,length(BoundaryFiles));
  %% Boundary file
```

```
BoundaryFile = fullfile(BoundaryFiles(n).folder,...
                    BoundaryFiles(n).name);
%% Extract embryo name
embryoName = erase(BoundaryFiles(n).name,...
                '-Boundary_SuperEmbryo.csv');
EmbryoNames{n} = embryoName;
fprintf('%s\n',embryoName);
%% Find matching MATLAB results folder
resultFolder = dir(fullfile(ResultsRoot,[embryoName '*']));
if isempty(resultFolder)
    warning('No MATLAB results folder found for %s',embryoName);
    continue
end
resultFolder = fullfile(resultFolder(1).folder,...
                    resultFolder(1).name);
%% Spectral file
SpectralFile = fullfile(resultFolder,...
                    'tables',...
                    'singlecell_results_final.csv');
if ~isfile(SpectralFile)
    warning('Missing spectral file for %s',embryoName);
    continue
end
%% Load data
Boundary = readtable(BoundaryFile);
Features = readtable(SpectralFile);
%% Check IDs
fprintf('Boundary cells: %d\n',height(Boundary));
fprintf('Spectral cells: %d\n',height(Features));
matchedIDs = intersect(Boundary.Calcium_ID,...
                   Features.CellID);
fprintf('Matched cells: %d\n',length(matchedIDs));
%% Merge
Merged = innerjoin(Boundary,Features,...
    'LeftKeys','Calcium_ID',...
    'RightKeys','CellID');
fprintf('Merged cells: %d\n',height(Merged));
%% Store merged table
AllMerged{n} = Merged;
%% Collect spatial coordinates
x = Merged.Centroid_1;
y = Merged.Centroid_2;
validXY = isfinite(x) & isfinite(y);
AllX = [AllX; x(validXY)];
```

```
    AllY = [AllY; y(validXY)];
    %% Collect feature values
    availableFeatures = featuresToPlot( ...
        ismember(featuresToPlot,...
        Merged.Properties.VariableNames));
    for k = 1:numel(availableFeatures)
        featureName = availableFeatures{k};
        values = Merged.(featureName);
        values = values(isfinite(values));
        if ~isfield(AllFeatureValues,featureName)
            AllFeatureValues.(featureName) = [];
        end
        AllFeatureValues.(featureName) = ...
            [AllFeatureValues.(featureName); values];
    end
end
%% ================================================================
%  GLOBAL PLOT LIMITS
%  ================================================================
%% Spatial limits
% These are identical for every embryo.
%
xMargin = 0.02 * (max(AllX) - min(AllX));
yMargin = 0.02 * (max(AllY) - min(AllY));
plotLimits.X = [min(AllX)-xMargin, max(AllX)+xMargin];
plotLimits.Y = [min(AllY)-yMargin, max(AllY)+yMargin];
fprintf('\n=======================================\n');
fprintf('GLOBAL SPATIAL LIMITS\n');
fprintf('X: %.3f to %.3f\n',plotLimits.X(1),plotLimits.X(2));
fprintf('Y: %.3f to %.3f\n',plotLimits.Y(1),plotLimits.Y(2));
%% Feature color limits
% Use 1st-99th percentile to prevent extreme outliers
% from dominating the color scale.
for k = 1:numel(featuresToPlot)
    featureName = featuresToPlot{k};
    if ~isfield(AllFeatureValues,featureName)
        continue
    end
    values = AllFeatureValues.(featureName);
    if isempty(values)
        continue
    end
    plotLimits.(featureName) = ...
        prctile(values,[1 99]);
```

```
    fprintf('%s: %.4f to %.4f\n',...
        featureName,...
        plotLimits.(featureName)(1),...
        plotLimits.(featureName)(2));
end
%% ========================================================================
%  SECOND PASS:
%  Generate plots using standardized limits
%  ========================================================================
for n = 1:length(BoundaryFiles)
    Merged = AllMerged{n};
    %% Skip embryos that could not be loaded
    if isempty(Merged)
        continue
    end
    embryoName = EmbryoNames{n};
    fprintf('\n=======================================\n');
    fprintf('Plotting embryo %d of %d\n',n,length(BoundaryFiles));
    fprintf('%s\n',embryoName);
    %% Find output folder again
    resultFolder = dir(fullfile(ResultsRoot,[embryoName '*']));
    if isempty(resultFolder)
        continue
    end
    OutputRoot = fullfile(resultFolder(1).folder,...
                          resultFolder(1).name);
    %% Save merged table
    OutputName = fullfile(OutputRoot,...
        [embryoName '_Spectral_Mapped.csv']);
    writetable(Merged,OutputName);
    %% Available features
    availableFeatures = featuresToPlot( ...
        ismember(featuresToPlot,...
        Merged.Properties.VariableNames));
    %% Plot each feature
    for k = 1:numel(availableFeatures)
        FeatureToPlot = availableFeatures{k};
        values = Merged.(FeatureToPlot);
        %% Create figure
        f = figure('Visible','off',...
                   'Units','pixels',...
                   'Position',[100 100 800 600],...
                   'Color','white');
        %% Plot
```

```
    scatter(Merged.Centroid_1,...
            Merged.Centroid_2,...
            25,...
            values,...
            'filled');
    %% Standardized spatial axes
    xlim(plotLimits.X);
    ylim(plotLimits.Y);
    %% Equal spatial scaling
    axis equal
    %% Reapply limits after axis equal
    xlim(plotLimits.X);
    ylim(plotLimits.Y);
    %% Standardized color scale
    clim(plotLimits.(FeatureToPlot));
    %% Colorbar
    cb = colorbar;
    cb.Label.String = FeatureToPlot;
    %% Labels
    title(sprintf('%s — %s',...
        embryoName,FeatureToPlot),...
        'Interpreter','none');
    xlabel('Template X');
    ylabel('Template Y');
    %% Consistent formatting
    set(gca,...
        'FontSize',12,...
        'LineWidth',1,...
        'Box','on');
    %% Output filename
    outPNG = fullfile(OutputRoot,...
        sprintf('%s_Mapped_%s.png',...
        embryoName,FeatureToPlot));
    %% Export
    try
        exportgraphics(f,outPNG,...
            'BackgroundColor','white',...
            'Resolution',300);
    catch
        saveas(f,outPNG);
    end
    close(f)
end
fprintf('Finished %s\n',embryoName);
```

```
end
fprintf('\n=======================================\n');
fprintf('All embryos complete.\n');
```

Combines all the individual embryo PNGs for each spectral feature into 6 pdfs

```
%% ================================================================
%  COMBINE SINGLE-EMBRYO MAPPED PLOTS BY FEATURE
%  Creates one PDF per spectral feature.
%
%  Each PDF is divided into:
%      1. Stage 13
%      2. Stage 13 Devitelined
%      3. Stage 14
%      4. Stage 14 Devitelined
%
%  Group assignment comes from the authoritative embryo table below.
%
%  Input structure:
%  E:\LAB\MATLAB_Results\
%      201020SP_E3_st_14\
%          201020SP_E3_Mapped_DominantFreqHz.png
%          201020SP_E3_Mapped_SpectralCentroid.png
%          ...
%      201020SP_E4_st_14_devitelined\
%          201020SP_E4_Mapped_DominantFreqHz.png
%          ...
% ================================================================
clear;
clc;
close all;
%% ========================= USER SETTINGS =========================
RootFolder = 'E:\LAB\MATLAB_Results';
OutputFolder = fullfile( ...
  RootFolder, ...
  'Combined_Feature_PDFs');
if ~exist(OutputFolder, 'dir')
  mkdir(OutputFolder);
end
%% ========================= FEATURES ==============================
Features = { ...
  'DominantFreqHz', ...
  'SpectralCentroid', ...
  'SpectralSpread', ...
```

```
    'SpectralEntropy', ...
    'TotalBandPower', ...
    'PeakPowerFraction'};
%% ==================== EMBRYO GROUP TABLE ==========================
% These are the authoritative experimental group assignments.
% The order here is preserved in the resulting PDFs.
Stage13 = { ...
    '180118SP_E7'
    '180118SP_E8'
    '180118SP_E9'
    '180125SP_E2'
    '180125SP_E3'
    '180216SP_E1'
    '180304SP_E4'
    '180304SP_E5'
    '180315SP_E1'
    '180315SP_E2'
    '180328SP_E6'
    '180328SP_E7'
    '180405SP_E3'
    '180405SP_E4'
    '180413SP_E3'
    '180427SP_E4'
    '180607SP_E5'
    '180612SP_E2'
    '180612SP_E3'
    '180622SP_E1'
    '180622SP_E6'
    '180628SP_E1'
    '180628SP_E4'
    '180703SP_E2'
    '180703SP_E5'
    '180706SP_E5'
    '180706SP_E6'
    '180802SP_E1'
    '180905SP_E2'
    '180905SP_E5'
    '180905SP_E8'
    '180911SP_E4'
    '190213SP_E2'
    '190918SP_E3'
    '191020SP_E1'
    '191020SP_E2'
    '191020SP_E5'
```

```
    '191030SP_E2'
    '191031SP_E1'
    '200114SP_E1'
    '200114SP_E2'
    '200115SP_E1'
    '200217SP_E3'
    '201007SP_E3'
    '201011SP_E1'
    '201023SP_E1'};
  Stage13Dev = { ...
    '180509SP_E7'
    '180731SP_E1'
    '180731SP_E2'
    '180731SP_E3'
    '180731SP_E4'
    '180928SP_E7'
    '181002SP_E7'
    '181002SP_E8'
    '181002SP_E9'
    '181005SP_E2'
    '190208SP_E1'
    '190411SP_E1'
    '190411SP_E2'
    '190411SP_E3'
    '190705SP_E4'
    '190716SP_E1'
    '190717SP_E1'
    '190717SP_E3'
    '190724SP_E2'
    '190724SP_E3'
    '190729SP_E1'
    '190730SP_E7'
    '190831SP_E4'
    '191030SPRN_E3'
    '191103SP_E1'
    '200202SP_E2'
    '200217SP_E2'
    '200301SP_E1'
    '200930SP_E1'
    '200930SP_E5'
    '201001SP_E6'
    '201003SP_E1'
    '201003SP_E2'
    '201003SP_E3'
```

```
    '201003SP_E4'
    '201003SP_E7'
    '201004SP_E5'
    '201007SP_E1'
    '201007SP_E2'
    '201007SP_E6'
    '201014SP_E6'
    '201019SP_E2'
    '201019SP_E3'
    '201019SP_E6'
    '201020SP_E2'
    '201023SP_E4'};
  Stage14 = { ...
    '180209SP_E4'
    '180209SP_E5'
    '180304SP_E7'
    '180315SP_E3'
    '180324SP_E1'
    '180328SP_E3'
    '180509SP_E6'
    '180622SP_E2'
    '180628SP_E5'
    '180703SP_E6'
    '180703SP_E7'
    '180711SP_E1'
    '180711SP_E5'
    '180802SP_E2'
    '180802SP_E6'
    '180905SP_E3'
    '180905SP_E4'
    '190206SP_E1'
    '190213SP_E5'
    '190320SP_E1'
    '190320SP_E6'
    '190730SP_E2'
    '190918SP_E4'
    '190918SP_E6'
    '191020RNSP_E6'
    '191020SP_E3'
    '191103SP_E2'
    '191104RN_E2'
    '200114SP_E3'
    '200115SP_E2'
    '200302RN_E5'
```

```
    '200313SP_E1'
    '200313SP_E2'
    '201004SP_E4'
    '201007SP_E5'
    '201007SP_E7'
    '201011SP_E5'
    '201014SP_E2'
    '201014SP_E4'
    '201015RN_E3'
    '201015SP_E4'
    '201019SP_E8'
    '201019SP_E9'
    '201020SP_E4'
    '201023SP_E2'
    '201023SP_E5'};
  Stage14Dev = { ...
    '180125SP_E4'
    '180802SP_E5'
    '180921SP_E1'
    '181002SP_E3'
    '181002SP_E4'
    '190327SP_E1'
    '190327SP_E6'
    '190327SP_E7'
    '190401SP_E3'
    '190619SP_E2'
    '190724SP_E5'
    '190724SP_E6'
    '190730SP_E3'
    '190730SP_E4'
    '190730SP_E5'
    '190730SP_E6'
    '190730SP_E8'
    '190918RN_E1'
    '190918SP_E5'
    '200202SP_E4'
    '200202SP_E5'
    '200202SP_E6'
    '200217SP_E5'
    '200301SP_E2'
    '200930SP_E2'
    '200930SP_E3'
    '200930SP_E8'
    '201001SP_E7'
```

```
    '201003SP_E5'
    '201003SP_E6'
    '201007SP_E4'
    '201007SP_E8'
    '201011SP_E2'
    '201014SP_E1'
    '201014SP_E7'
    '201015SP_E5'
    '201019SP_E1'
    '201019SP_E4'
    '201020SP_E1'
    '201020SP_E3'
    '201024SP_E1'};
%% ======================== GROUP STRUCTURE ===========================
GroupNames = { ...
    'Stage 13', ...
    'Stage 13 Devitelined', ...
    'Stage 14', ...
    'Stage 14 Devitelined'};
GroupEmbryos = { ...
    Stage13, ...
    Stage13Dev, ...
    Stage14, ...
    Stage14Dev};
%% ======================== GROUP SUMMARY ===========================
fprintf('\n');
fprintf('=============================================================\n');
fprintf('EMBRYO GROUPING\n');
fprintf('=============================================================\n');
for g = 1:length(GroupNames)
    fprintf('%-25s %3d embryos\n', ...
        GroupNames{g}, ...
        length(GroupEmbryos{g}));
end
TotalEmbryos = sum(cellfun(@length, GroupEmbryos));
fprintf('-------------------------------------------------------------\n');
fprintf('%-25s %3d embryos\n', ...
    'TOTAL', ...
    TotalEmbryos);
fprintf('=============================================================\n');
fprintf('\n');
%% ===================== FIND ALL FOLDERS ===========================
FolderInfo = dir(RootFolder);
FolderInfo = FolderInfo([FolderInfo.isdir]);
```

```
FolderInfo = FolderInfo(~ismember( ...
  {FolderInfo.name}, ...
  {'.', '..'}));
fprintf('Searching for embryo folders...\n');
fprintf('Root: %s\n\n', RootFolder);
%% ====================== MAIN FEATURE LOOP ========================
for f = 1:length(Features)
  FeatureName = Features{f};
  fprintf('\n');
  fprintf('================================================================\n');
  fprintf('PROCESSING FEATURE: %s\n', FeatureName);
  fprintf('================================================================\n');
  %% ------------------------------------------------------------
  % Output PDF
  % ------------------------------------------------------------
  OutputPDF = fullfile( ...
    OutputFolder, ...
    [FeatureName '_AllEmbryos.pdf']);
  % Delete existing PDF
  if exist(OutputPDF, 'file')
    delete(OutputPDF);
  end
  FirstPage = true;
  %% ===================== GROUP LOOP ============================
  for g = 1:length(GroupNames)
    GroupName = GroupNames{g};
    EmbryoList = GroupEmbryos{g};
    fprintf('\n');
    fprintf('%s (%d embryos)\n', ...
      GroupName, ...
      length(EmbryoList));
    %% ------------------------------------------------------
    % Locate plots for this group
    % ------------------------------------------------------
    PlotFiles = {};
    PlotNames = {};
    for e = 1:length(EmbryoList)
      EmbryoID = EmbryoList{e};
      %% ----------------------------------------------
      % Find folder associated with this embryo
      %
      % Example:
      %
      % EmbryoID:
```

```
%   200217SP_E3
%
% Folder:
%   200217SP_E3_st_13
%
% or:
%   200217SP_E3_st_13_albino
%
% ------------------------------------------------------
Matches = startsWith( ...
    {FolderInfo.name}, ...
    [EmbryoID '_']);
MatchingFolders = FolderInfo(Matches);
if isempty(MatchingFolders)
    fprintf('  MISSING FOLDER: %s\n', ...
        EmbryoID);
    continue;
end
%% ------------------------------------------------------
% Check for duplicate folders
% ------------------------------------------------------
if length(MatchingFolders) > 1
    fprintf(['  WARNING: Multiple folders found for ' ...
        '%s. Using: %s\n'], ...
        EmbryoID, ...
        MatchingFolders(1).name);
end
EmbryoFolder = fullfile( ...
    RootFolder, ...
    MatchingFolders(1).name);
%% ------------------------------------------------------
% Construct expected plot filename
% ------------------------------------------------------
PlotBaseName = [ ...
    EmbryoID ...
    '_Mapped_' ...
    FeatureName];
%% Search supported extensions
% ------------------------------------------------------
CandidateExtensions = { ...
    '.png'
    '.jpg'
    '.jpeg'
    '.tif'
```

```
        '.tiff'
        '.fig'};
    FoundPlot = '';
    for x = 1:length(CandidateExtensions)
        Candidate = fullfile( ...
            EmbryoFolder, ...
            [PlotBaseName CandidateExtensions{x}]);
        if exist(Candidate, 'file')
            FoundPlot = Candidate;
            break;
        end
    end
    %% Record result
    % -----------------------------------------------------
    if isempty(FoundPlot)
        fprintf('  MISSING PLOT: %s\n', ...
            PlotBaseName);
    else
        fprintf('  Found: %s\n', ...
            EmbryoID);
        PlotFiles{end+1} = FoundPlot;
        PlotNames{end+1} = EmbryoID;
    end
end
%% ================== NOTHING FOUND ==========================
if isempty(PlotFiles)
    fprintf('  No plots found -- skipping group.\n');
    continue;
end
%% ===================== PAGE SETUP ==========================
% 4 x 4 = 16 plots per page
nRows = 4;
nCols = 4;
PlotsPerPage = nRows * nCols;
nPages = ceil(length(PlotFiles) / PlotsPerPage);
%% ===================== PAGE LOOP ===========================
for p = 1:nPages
    StartIdx = (p-1)*PlotsPerPage + 1;
    EndIdx = min( ...
        p*PlotsPerPage, ...
        length(PlotFiles));
    CurrentIdx = StartIdx:EndIdx;
    %% -----------------------------------------------------
    % Create figure
```

```
% ------------------------------------------------
fig = figure( ...
    'Color', 'w', ...
    'Units', 'normalized', ...
    'Position', [0.02 0.04 0.96 0.90], ...
    'Visible', 'off');
%% ------------------------------------------------
% Page title
% ------------------------------------------------
sgtitle( ...
    sprintf('%s — %s', ...
    FeatureName, ...
    GroupName), ...
    'FontSize', 18, ...
    'FontWeight', 'bold');
%% ------------------------------------------------
% Tiled layout
% ------------------------------------------------
tiledlayout( ...
    nRows, ...
    nCols, ...
    'TileSpacing', 'compact', ...
    'Padding', 'compact');
%% ------------------------------------------------
% Add plots
% ------------------------------------------------
for j = 1:length(CurrentIdx)
    idx = CurrentIdx(j);
    nexttile;
    PlotFile = PlotFiles{idx};
    [~, ~, ext] = fileparts(PlotFile);
    %% ---------------------------------------------
    % Image files
    % ---------------------------------------------
    if ismember( ...
            lower(ext), ...
            {'.png','.jpg','.jpeg','.tif','.tiff'})
        img = imread(PlotFile);
        imshow(img);
        axis image off;
    %% ---------------------------------------------
    % MATLAB FIG files
    % ---------------------------------------------
    elseif strcmpi(ext, '.fig')
```

```
        TempFig = openfig( ...
            PlotFile, ...
            'invisible');
        AxesHandles = findobj( ...
            TempFig, ...
            'Type', ...
            'axes');
        if ~isempty(AxesHandles)
            SourceAxes = AxesHandles(1);
            copyobj( ...
                allchild(SourceAxes), ...
                gca);
            xlim(gca, SourceAxes.XLim);
            ylim(gca, SourceAxes.YLim);
        end
        close(TempFig);
    end
    %% ------------------------------------------------
    % Embryo label
    % ------------------------------------------------
    title( ...
        strrep( ...
        PlotNames{idx}, ...
        '_', ...
        '\_'), ...
        'Interpreter', ...
        'tex', ...
        'FontSize', ...
        9);
end
%% ================= PAGE NUMBER ========================
if nPages > 1
    annotation( ...
        fig, ...
        'textbox', ...
        [0.84 0.005 0.14 0.025], ...
        'String', ...
        sprintf('Page %d of %d', ...
        p, ...
        nPages), ...
        'EdgeColor', ...
        'none', ...
        'HorizontalAlignment', ...
        'right', ...
```

```
                'FontSize', ...
                9);
        end
        %% ================ EXPORT PAGE ========================
        if FirstPage
            exportgraphics( ...
                fig, ...
                OutputPDF, ...
                'ContentType', ...
                'image', ...
                'Resolution', ...
                300);
            FirstPage = false;
        else
            exportgraphics( ...
                fig, ...
                OutputPDF, ...
                'ContentType', ...
                'image', ...
                'Resolution', ...
                300, ...
                'Append', ...
                true);
        end
        close(fig);
    end
  end
  fprintf('\nCreated:\n%s\n', OutputPDF);
end
%% ============================ DONE ================================
fprintf('\n');
fprintf('============================================================\n');
fprintf('ALL PDFs CREATED\n');
fprintf('============================================================\n');
fprintf('Output folder:\n%s\n\n', OutputFolder);
for f = 1:length(Features)
  fprintf('  %s_AllEmbryos.pdf\n', ...
      Features{f});
end
fprintf('\n');
```

3. Group mapping:

Created 4 paneled figures (a panel for each group) for the [Spectral feature]_GroupSpatialMaps, [Spectral feature]_PopulationMedian_GroupSpatialMaps, [Spectral feature]_PopulationSD_GroupSpatialMaps, [Spectral feature]_SpatialCoverage_GroupSpatialMaps and a CompleteCNPUGrid_QC

The spatial analysis operates at two hierarchical levels: cell → embryo x CNPU → developmental-group CNPU
For each feature, cells are first summarized within each embryo and CNPU. The resulting embryo-level CNPU values are then summarized across embryos within each experimental group. This prevents embryos containing more cells from automatically contributing more weight to the group-level spatial estimate.

Population central tendency
For each CNPU, the group-level central tendency is calculated as the median of the embryo-level CNPU feature values.

Thus, if $x_{e,c}$ is the feature value for embryo e in CNPU c: $\tilde{x}_c = \mathrm{median}_e(x_{e,c})$
This provides an embryo-balanced estimate of the typical feature value at each spatial location.

Population variability
The standard deviation across the embryo-level CNPU values is also calculated:

$$SD_c = \sqrt{\frac{\sum_e (x_{e,c} - \bar{x}_c)^2}{N_c - 1}}$$

This quantifies between-embryo variability in the feature at each spatial location.

Spatial coverage
The analysis additionally calculates the proportion of embryos contributing data to each CNPU:
$\mathrm{Coverage}_c = (N_{\mathrm{embryos},c}/N_{\mathrm{embryos,group}}) \times 100$
Thus, the spatial coverage map provides a measure of how consistently a spatial location is represented across embryos rather than interpreting missing observations as biological values.

Spatial Map Generation
For every spectral feature, the mapping pipeline produces three complementary maps:
Population central tendency — embryo-balanced median feature value.
Population variability — standard deviation of embryo-level CNPU values.
Spatial coverage — percentage of embryos contributing to each CNPU.
The code explicitly defines these three outputs.
All experimental groups are displayed on the same spatial canvas. The feature values are plotted at CNPU center coordinates, and a common color scale is applied across groups. The Stage 13 or Stage 14 Sox2 boundary is then overlaid according to the developmental group.

The common color scale is calculated from the observed group-level median values across the groups being compared. This prevents the same color from representing different numerical values in different experimental groups.

Driver:

```
% GROUP-LEVEL SPATIAL SPECTRAL ANALYSIS DRIVER SCRIPT
% Workflow:
%   1. Load configuration
%   2. Resolve one source file per biological embryo
%   3. Load and standardize mapped spectral data
%   4. Assign cells to fixed 9x9 legacy CNPUs
%   5. Create complete analysis CNPU grid
%   6. Aggregate embryo-balanced group statistics
%   7. Calculate shared figure color limits
%   8. Generate QC and primary spatial maps
%   9. Save results
%% ======================================================================
clear; clc; close all;
%% ======================================================================
% 1. CONFIGURATION
%% ======================================================================
cfg = getGroupSpatialConfig();
fprintf('GROUP SPATIAL SPECTRAL ANALYSIS\n');
fprintf('==============================================================\n');
%% ======================================================================
% 2. RESOLVE MAPPED EMBRYO FILES
%% ======================================================================
[MappedFiles, EmbryoInfo] = ...
  resolveMappedEmbryos(cfg);
%% ======================================================================
% 3. LOAD AND STANDARDIZE CELL-LEVEL DATA
%% ======================================================================
AllCells = ...
  loadMappedEmbryoData(...
  MappedFiles,...
  EmbryoInfo,...
  cfg);
%% ======================================================================
% 4. ASSIGN CELLS TO LEGACY 9x9 CNPUs
%% ======================================================================
AllCells = ...
  assignCellsToCNPUs(...
  AllCells,...
  cfg);
```

```
%% ======================================================================
% 5. CREATE COMPLETE ANALYSIS GRID
%% ======================================================================
FullGrid = ...
  createCompleteCNPUGrid(...
  AllCells,...
  cfg);
%% ======================================================================
% 6. SAVE CELL-LEVEL CNPU DATA
%% ======================================================================
CellCNPUFile = fullfile(...
  cfg.OutputRoot,...
  'All_MappedCells_with_CNPU.csv');
writetable(...
  AllCells,...
  CellCNPUFile);
fprintf('\nSaved cell-to-CNPU table:\n%s\n',...
  CellCNPUFile);
%% ======================================================================
% 7. COMPLETE-GRID QC
%% ======================================================================
plotCompleteGridQC(...
  FullGrid,...
  cfg);
%% ======================================================================
% 8. GROUP-LEVEL AGGREGATION
%% ======================================================================
Results = ...
  aggregateGroupCNPU(...
  AllCells,...
  FullGrid,...
  EmbryoInfo,...
  cfg);
%% ======================================================================
% 9. SHARED FIGURE COLOR LIMITS
%% ======================================================================
ColorLimits = ...
  calculateSharedColorLimits(...
  Results,...
  cfg);
%% ======================================================================
% 10. PRIMARY SPATIAL FIGURES
%% ======================================================================
plotGroupSpatialMaps(...
```

```
    Results,...
    FullGrid,...
    ColorLimits,...
    cfg);
%% ======================================================================
% 11. SAVE ALL RESULTS
%% ======================================================================
save(...
    fullfile(...
    cfg.OutputRoot,...
    'GroupSpatial_Spectral_Results.mat'),...
    'Results',...
    'FullGrid',...
    'ColorLimits',...
    'cfg',...
    '-v7.3');
fprintf('\n============================================================\n');
fprintf('ANALYSIS COMPLETE\n');
fprintf('============================================================\n');
fprintf('\nOutput directory:\n%s\n',...
    cfg.OutputRoot);
```

Function 1: getGroupSpatialConfig

```
function cfg = getGroupSpatialConfig()
%% ======================================================================
% GET GROUP SPATIAL ANALYSIS CONFIGURATION
%% ======================================================================
% Paths
%% ----------------------------------------------------------------
cfg.MappedRoot = ...
    'E:\LAB\MATLAB_Results';
cfg.MappedPattern = ...
    '*_Spectral_Mapped.csv';
cfg.OutputRoot = ...
    'E:\LAB\GroupSpatialAnalysis';
cfg.BoundaryRoot = ...
    'D:\LAB\SuperEmbryoBoundaries';
cfg.MatrixFormFile = ...

'D:\LAB\SuperEmbryoBoundaries\SuperEmbryo_MatrixForm\1_SuperEmbryo_MatrixFormTh0_
OnlySpike.csv';
%% Create output directory
if ~isfolder(cfg.OutputRoot)
```

```
    mkdir(cfg.OutputRoot);
end
%% ------------------------------------------------------------
% Spectral features
%% ------------------------------------------------------------
cfg.Features = { ...
    'DominantFreqHz',...
    'SpectralEntropy',...
    'SpectralCentroid',...
    'SpectralSpread',...
    'TotalBandPower',...
    'PeakPowerFraction'};
%% ------------------------------------------------------------
% Experimental groups
%% ------------------------------------------------------------
cfg.AllowedGroups = [ ...
    "Stage13",...
    "Stage13_devitelined",...
    "Stage14",...
    "Stage14_devitelined"];
%% ------------------------------------------------------------
% Group assignment
%% ------------------------------------------------------------
cfg.GroupAssignmentFile = ...
    fullfile(...
    cfg.OutputRoot,...
    'Embryo_Group_Assignments.csv');
%% ------------------------------------------------------------
% Legacy CNPU geometry
%% ------------------------------------------------------------
cfg.CNPUsize = 9;
cfg.GridOriginX = 120;
cfg.GridOriginY = 1;
cfg.ExpectedXRange = [120 1122];
cfg.ExpectedYRange = [1 1190];
%% ------------------------------------------------------------
% Legacy smoothing
%% ------------------------------------------------------------
cfg.DoSmoothing = true;
cfg.SmoothWindow = 7;
%% ------------------------------------------------------------
% Analysis mode
%
% test = one group + one feature
```

```
% full = all groups + all features
%% ------------------------------------------------------------
cfg.TestMode = false;
cfg.TestGroup = "Stage13";
cfg.TestFeature = "SpectralEntropy";
%% ------------------------------------------------------------
% Sox2 boundary files
%
% Biological Stage 13 uses Stage 14 template boundary.
% Biological Stage 14 uses Stage 15 template boundary.
%% ------------------------------------------------------------
cfg.Sox2BoundaryFiles.Stage13 = ...
  fullfile(...
  cfg.BoundaryRoot,...
  '1_new_Stage_14_Sox2_Boundary.csv');
cfg.Sox2BoundaryFiles.Stage14 = ...
  fullfile(...
  cfg.BoundaryRoot,...
  '1_new_Stage_15_Sox2_Boundary.csv');
%% ------------------------------------------------------------
% Boundary plotting
%% ------------------------------------------------------------
cfg.BoundaryLineStyle = ':';
cfg.BoundaryLineWidth = 2;
%% ------------------------------------------------------------
% Figure settings
%% ------------------------------------------------------------
cfg.FigurePosition = [100 100 900 700];
cfg.FigureResolution = 300;
end
```

Function 2: resolveMappedEmbryos

```
function [MappedFiles, EmbryoInfo] = ...
  resolveMappedEmbryos(cfg)
%% ============================================================
% RESOLVE MAPPED EMBRYO FILES
% Returns exactly one mapped source file per biological embryo.
% Special case:
%   181002SP_E8
%   181002SP_E8_st13_devitelined_
% are treated as one biological embryo.
% When both exist, the explicitly devitelined source is selected.
```

```
%% ================================================================
MappedFiles = dir(fullfile(...
   cfg.MappedRoot,...
   '**',...
   cfg.MappedPattern));
nRawFiles = numel(MappedFiles);
fprintf('\n============================================================\n');
fprintf('MAPPED EMBRYO FILES\n');
fprintf('============================================================\n');
fprintf('Raw mapped files found: %d\n',...
   nRawFiles);
if nRawFiles == 0
   error('No mapped spectral files found.');
end
%% ----------------------------------------------------------------
% Extract raw filename-derived IDs
%% ----------------------------------------------------------------
RawEmbryoNames = strings(nRawFiles,1);
for n = 1:nRawFiles
   RawEmbryoNames(n) = erase(...
      MappedFiles(n).name,...
      '_Spectral_Mapped.csv');
end
%% ----------------------------------------------------------------
% Canonical biological IDs
%% ----------------------------------------------------------------
CanonicalEmbryoNames = regexprep(...
   RawEmbryoNames,...
   '_st13_devitelined_$',...
   '',...
   'ignorecase');
[UniqueEmbryoNames,~,canonicalIndex] = ...
   unique(...
   CanonicalEmbryoNames,...
   'stable');
nBiologicalEmbryos = ...
   numel(UniqueEmbryoNames);
fprintf(...
   'Biological embryos represented: %d\n',...
   nBiologicalEmbryos);
%% ----------------------------------------------------------------
% Choose one file per biological embryo
%% ----------------------------------------------------------------
SelectedFileIndex = ...
```

```
    zeros(nBiologicalEmbryos,1);
for k = 1:nBiologicalEmbryos
    candidateIdx = find(...
        canonicalIndex == k);
    if isscalar(candidateIdx)
        SelectedFileIndex(k) = ...
            candidateIdx;
        continue;
    end
    %% Prefer explicit devitelined source
    devMask = contains(...
        RawEmbryoNames(candidateIdx),...
        '_st13_devitelined_',...
        'IgnoreCase',true);
    devCandidates = ...
        candidateIdx(devMask);
    if isscalar(devCandidates)
        SelectedFileIndex(k) = ...
            devCandidates;
    elseif isempty(devCandidates)
        error([...
            'Multiple mapped files found for biological ',...
            'embryo %s with no unique preferred source.'],...
            UniqueEmbryoNames(k));
    else
        error([...
            'Multiple devitelined mapped files found for ',...
            'biological embryo %s.'],...
            UniqueEmbryoNames(k));
    end
end
%% ------------------------------------------------------------
% Retain selected source files
%% ------------------------------------------------------------
MappedFiles = ...
    MappedFiles(SelectedFileIndex);
SelectedSourceNames = ...
    RawEmbryoNames(SelectedFileIndex);
EmbryoInfo = table(...
    UniqueEmbryoNames,...
    SelectedSourceNames,...
    'VariableNames',{...
    'Embryo',...
    'SourceFile'});
```

```
fprintf(...
  'Mapped files retained for analysis: %d\n',...
  numel(MappedFiles));
%% ------------------------------------------------------------
% Report duplicate resolution
%% ------------------------------------------------------------
fprintf('\n============================================================\n');
fprintf('FILE RESOLUTION QC\n');
fprintf('============================================================\n');
for k = 1:nBiologicalEmbryos
  candidateIdx = find(...
    canonicalIndex == k);
  if numel(candidateIdx) > 1
    fprintf('\nBiological embryo: %s\n',...
      UniqueEmbryoNames(k));
    fprintf('  Candidate source files:\n');
    for j = 1:numel(candidateIdx)
      fprintf(...
        '    %s\n',...
        RawEmbryoNames(candidateIdx(j)));
    end
    fprintf('  Selected source file:\n');
    fprintf(...
      '    %s\n',...
      SelectedSourceNames(k));
  end
end
%% ------------------------------------------------------------
% Expected embryo count
%% ------------------------------------------------------------
if numel(MappedFiles) ~= 179
  warning(...
    'Expected 179 biological embryos; found %d.',...
    numel(MappedFiles));
end
end
```

Function 3: loadMappedEmbryoData

```
function AllCells = ...
  loadMappedEmbryoData(...
  MappedFiles,...
  EmbryoInfo,...
  cfg)
```

```
%% ======================================================================
% LOAD AND STANDARDIZE MAPPED EMBRYO DATA
% Loads one resolved mapped source file per biological embryo,
% attaches biological group metadata, and standardizes the table
% structure used by the downstream CNPU pipeline.
%% ======================================================================
nEmbryos = numel(MappedFiles);
%% ======================================================================
% 1. READ GROUP ASSIGNMENTS
%% ======================================================================
Assignments = readtable(...
  cfg.GroupAssignmentFile,...
  'TextType','string');
if ~all(ismember(...
      {'Embryo','Group'},...
      Assignments.Properties.VariableNames))
  error(...
      'Group assignment file must contain Embryo and Group.');
end
Assignments.Embryo = ...
  string(Assignments.Embryo);
Assignments.Group = ...
  string(Assignments.Group);
%% ======================================================================
% 2. VALIDATE GROUP LABELS
%% ======================================================================
invalidGroups = setdiff(...
  unique(Assignments.Group),...
  cfg.AllowedGroups);
invalidGroups(invalidGroups=="") = [];
if ~isempty(invalidGroups)
  error(...
      'Invalid group labels: %s',...
      strjoin(invalidGroups,', '));
end
%% ======================================================================
% 3. VALIDATE EMBRYO ASSIGNMENTS
%% ======================================================================
[isAssigned,assignmentIdx] = ...
  ismember(...
  EmbryoInfo.Embryo,...
  Assignments.Embryo);
if any(~isAssigned)
  fprintf('\nEmbryos without group assignments:\n');
```

```
    disp(...
        EmbryoInfo.Embryo(~isAssigned));
    error(...
        'Every biological embryo must have a group.');
end
%% ------------------------------------------------------------
% Check duplicate assignment rows
%% ------------------------------------------------------------
[uniqueAssigned,~,counts] = ...
    unique(Assignments.Embryo);
duplicateEmbryos = ...
    uniqueAssigned(...
    accumarray(counts,1) > 1);
if ~isempty(duplicateEmbryos)
    fprintf('\nDuplicate assignment rows detected for:\n');
    disp(duplicateEmbryos);
    error(...
        'Each biological embryo must appear exactly once in the group assignment file.');
end
%% ====================================================================
% 4. ATTACH GROUP TO EMBRYOINFO
%% ====================================================================
EmbryoInfo.Group = ...
    Assignments.Group(assignmentIdx);
%% ====================================================================
% 5. LOAD EACH EMBRYO
%% ====================================================================
AllData = cell(nEmbryos,1);
fprintf('\n============================================================\n');
fprintf('LOADING MAPPED EMBRYO DATA\n');
fprintf('============================================================\n');
for n = 1:nEmbryos
    filePath = fullfile(...
        MappedFiles(n).folder,...
        MappedFiles(n).name);
    embryoName = ...
        EmbryoInfo.Embryo(n);
    sourceName = ...
        EmbryoInfo.SourceFile(n);
    groupName = ...
        EmbryoInfo.Group(n);
    fprintf(...
        '%3d/%3d  %s  [%s]\n',...
        n,...
```

```
    nEmbryos,...
    embryoName,...
    groupName);
%% ------------------------------------------------------
% Read mapped table
%% ------------------------------------------------------
T = readtable(...
    filePath,...
    'VariableNamingRule','preserve');
%% ------------------------------------------------------
% Required coordinates
%% ------------------------------------------------------
if ~all(ismember(...
        {'Centroid_1','Centroid_2'},...
        T.Properties.VariableNames))
    warning(...
        '%s missing Centroid_1/2. Skipping.',...
        MappedFiles(n).name);
    continue;
end
%% ------------------------------------------------------
% Standardized variables
%% ------------------------------------------------------
requiredVars = {...
    'Centroid_1',...
    'Centroid_2'};
%% ------------------------------------------------------
% Preserve optional cell identifier
%% ------------------------------------------------------
if ismember(...
        'Calcium_ID',...
        T.Properties.VariableNames)
    requiredVars{end+1} = ...
        'Calcium_ID';
elseif ismember(...
        'CellID',...
        T.Properties.VariableNames)
    requiredVars{end+1} = ...
        'CellID';
end
%% ------------------------------------------------------
% Ensure all spectral features exist
%% ------------------------------------------------------
for k = 1:numel(cfg.Features)
```

```
    featureName = ...
        cfg.Features{k};
    if ~ismember(...
            featureName,...
            T.Properties.VariableNames)
        T.(featureName) = ...
            nan(height(T),1);
        warning(...
            '%s missing %s; filling with NaN.',...
            MappedFiles(n).name,...
            featureName);
    end
    requiredVars{end+1} = ...
        featureName;
end
%% ------------------------------------------------------
% Restrict to standardized variables
%% ------------------------------------------------------
T = T(:,requiredVars);
%% ------------------------------------------------------
% Add biological metadata
%% ------------------------------------------------------
T.Embryo = ...
    repmat(...
    embryoName,...
    height(T),1);
T.Group = ...
    repmat(...
    groupName,...
    height(T),1);
T.SourceFile = ...
    repmat(...
    sourceName,...
    height(T),1);
%% ------------------------------------------------------
% Put metadata first
%% ------------------------------------------------------
T = T(:,[...
    {'Embryo'},...
    {'Group'},...
    {'SourceFile'},...
    requiredVars(:)']);
%% ------------------------------------------------------
% Store
```

```
    %% ------------------------------------------------------
    AllData{n} = T;
end
%% =====================================================================
% 6. REMOVE FAILED LOADS
%% =====================================================================
AllData = ...
    AllData(~cellfun(@isempty,AllData));
if isempty(AllData)
    error(...
        'No usable mapped embryo tables were loaded.');
end
%% =====================================================================
% 7. CONCATENATE CELL-LEVEL DATA
%% =====================================================================
AllCells = ...
    vertcat(AllData{:});
fprintf('\nTotal mapped cells loaded: %d\n',...
    height(AllCells));
fprintf(...
    'Biological embryos loaded: %d\n',...
    numel(unique(AllCells.Embryo)));
fprintf(...
    'Source files loaded: %d\n',...
    numel(unique(AllCells.SourceFile)));
%% =====================================================================
% 8. GROUP QC
%% =====================================================================
fprintf('\n=========================================================\n');
fprintf('FINAL BIOLOGICAL EMBRYO COUNTS\n');
fprintf('=========================================================\n');
for g = 1:numel(cfg.AllowedGroups)
    groupName = ...
        cfg.AllowedGroups(g);
    n = sum(...
        EmbryoInfo.Group == groupName);
    fprintf(...
        '  %-25s %d embryos\n',...
        groupName,...
        n);
end
%% =====================================================================
% 9. CELL COUNTS BY GROUP
%% =====================================================================
```

```
fprintf('\n=========================================================\n');
fprintf('CELL COUNTS BY GROUP\n');
fprintf('=========================================================\n');
for g = 1:numel(cfg.AllowedGroups)
  groupName = ...
    cfg.AllowedGroups(g);
  nCells = sum(...
    AllCells.Group == groupName);
  fprintf(...
    '  %-25s %d cells\n',...
    groupName,...
    nCells);
end
end
```

Function 4: assignCellsToCNPUs

```
function AllCells = ...
  assignCellsToCNPUs(AllCells,cfg)
%% ====================================================================
% ASSIGN CELLS TO LEGACY 9x9 CNPUs
%% ====================================================================
fprintf('\n=========================================================\n');
fprintf('SPECTRAL MAPPED COORDINATE QC\n');
fprintf('=========================================================\n');
X = AllCells.Centroid_1;
Y = AllCells.Centroid_2;
validXY = ...
  isfinite(X) & ...
  isfinite(Y);
fprintf(...
  'Cells with valid X/Y: %d of %d (%.2f%%)\n',...
  sum(validXY),...
  numel(validXY),...
  100*mean(validXY));
mappedMinX = min(X(validXY));
mappedMaxX = max(X(validXY));
mappedMinY = min(Y(validXY));
mappedMaxY = max(Y(validXY));
fprintf(...
  'Observed spectral X range: %.6f to %.6f\n',...
  mappedMinX,...
  mappedMaxX);
fprintf(...
```

```
    'Observed spectral Y range: %.6f to %.6f\n',...
    mappedMinY,...
    mappedMaxY);
if mappedMinX < cfg.ExpectedXRange(1) || ...
   mappedMaxX > cfg.ExpectedXRange(2) || ...
   mappedMinY < cfg.ExpectedYRange(1) || ...
   mappedMaxY > cfg.ExpectedYRange(2)
    warning([...
        'Some spectral cells fall outside the populated ',...
        'legacy MatrixForm coordinate extent. ',...
        'These cells are retained for analysis.']);
end
AllCells = ...
    AllCells(validXY,:);
%% ------------------------------------------------------------
% CNPU assignment
%% ------------------------------------------------------------
fprintf('\n============================================================\n');
fprintf('ASSIGNING CELLS TO CNPUs\n');
fprintf('============================================================\n');
AllCells.CNPU_X = floor(...
    (AllCells.Centroid_1 - cfg.GridOriginX) ./ ...
    cfg.CNPUsize);
AllCells.CNPU_Y = floor(...
    (AllCells.Centroid_2 - cfg.GridOriginY) ./ ...
    cfg.CNPUsize);
AllCells.CNPU_CenterX = ...
    cfg.GridOriginX + ...
    AllCells.CNPU_X .* ...
    cfg.CNPUsize + ...
    cfg.CNPUsize/2;
AllCells.CNPU_CenterY = ...
    cfg.GridOriginY + ...
    AllCells.CNPU_Y .* ...
    cfg.CNPUsize + ...
    cfg.CNPUsize/2;
AllCells.CNPU_ID = ...
    string(AllCells.CNPU_X) + "_" + ...
    string(AllCells.CNPU_Y);
fprintf(...
    'Unique observed CNPUs represented: %d\n',...
    numel(unique(AllCells.CNPU_ID)));
end
```

Function 5: createCompleteCNPUGrid

```
function FullGrid = ...
  createCompleteCNPUGrid(AllCells,cfg)
%% ================================================================
% CREATE COMPLETE ANALYSIS CNPU GRID
% Creates one fixed CNPU lattice shared by all experimental groups.
% Geometry is defined ONLY by the verified legacy SuperEmbryo
% coordinate system:
%   CNPU size     = cfg.CNPUsize
%   Grid origin   = (cfg.GridOriginX,cfg.GridOriginY)
%   X extent      = cfg.ExpectedXRange
%   Y extent      = cfg.ExpectedYRange
%
% The Sox2 boundaries are NOT used to define grid extent.
% They are used only to mark which CNPUs fall inside the
% corresponding biological template.
%
% IMPORTANT:
%   This function does NOT assign cells.
%   Cell-to-CNPU assignment has already occurred in assignCellsToCNPUs.m.
%% ================================================================
fprintf('\n============================================================\n');
fprintf('CREATING COMPLETE CNPU GRID\n');
fprintf('============================================================\n');
%% ================================================================
% 1. VALIDATE LEGACY GEOMETRY
%% ================================================================
if cfg.CNPUsize <= 0
  error('cfg.CNPUsize must be > 0.');
end
if cfg.ExpectedXRange(1) > cfg.ExpectedXRange(2)
  error('cfg.ExpectedXRange must be [minX maxX].');
end
if cfg.ExpectedYRange(1) > cfg.ExpectedYRange(2)
  error('cfg.ExpectedYRange must be [minY maxY].');
end
%% ----------------------------------------------------------------
% Verify that the configured extent begins at the grid origin
%% ----------------------------------------------------------------
if abs(...
      cfg.ExpectedXRange(1) - cfg.GridOriginX) > 1e-9
  error([...
     'Expected X range does not begin at GridOriginX. ',...
     'Grid geometry is inconsistent.']);
```

```
end
if abs(...
      cfg.ExpectedYRange(1) - cfg.GridOriginY) > 1e-9
   error([...
      'Expected Y range does not begin at GridOriginY. ',...
      'Grid geometry is inconsistent.']);
end
%% ====================================================================
% 2. COMPLETE LEGACY GRID INDICES
%% ====================================================================
minCNPU_X = 0;
minCNPU_Y = 0;
maxCNPU_X = ceil(...
   (cfg.ExpectedXRange(2) - cfg.GridOriginX) ./ ...
   cfg.CNPUsize) - 1;
maxCNPU_Y = ceil(...
   (cfg.ExpectedYRange(2) - cfg.GridOriginY) ./ ...
   cfg.CNPUsize) - 1;
%% --------------------------------------------------------------
% Construct complete rectangular lattice
%% --------------------------------------------------------------
[gridX,gridY] = ndgrid(...
   minCNPU_X:maxCNPU_X,...
   minCNPU_Y:maxCNPU_Y);
CNPU_X = gridX(:);
CNPU_Y = gridY(:);
%% ====================================================================
% 3. CNPU SPATIAL COORDINATES
%% ====================================================================
CNPU_StartX = ...
   cfg.GridOriginX + ...
   CNPU_X .* cfg.CNPUsize;
CNPU_EndX = ...
   CNPU_StartX + ...
   cfg.CNPUsize;
CNPU_StartY = ...
   cfg.GridOriginY + ...
   CNPU_Y .* cfg.CNPUsize;
CNPU_EndY = ...
   CNPU_StartY + ...
   cfg.CNPUsize;
CNPU_CenterX = ...
   CNPU_StartX + ...
   cfg.CNPUsize/2;
```

```
CNPU_CenterY = ...
  CNPU_StartY + ...
  cfg.CNPUsize/2;
CNPU_ID = ...
  string(CNPU_X) + "_" + ...
  string(CNPU_Y);
%% ================================================================
% 4. BUILD COMPLETE GRID TABLE
%% ================================================================
FullGrid = table(...
  CNPU_X,...
  CNPU_Y,...
  CNPU_StartX,...
  CNPU_EndX,...
  CNPU_StartY,...
  CNPU_EndY,...
  CNPU_CenterX,...
  CNPU_CenterY,...
  CNPU_ID);
%% ================================================================
% 5. OBSERVED CELL INFORMATION
%% ================================================================
FullGrid.Observed = ...
  false(height(FullGrid),1);
FullGrid.NCells = ...
  zeros(height(FullGrid),1);
if ~isempty(AllCells)
  %% ---------------------------------------------------
  % Observed CNPU IDs
  %% ---------------------------------------------------
  validCNPU = ...
     ~ismissing(AllCells.CNPU_ID);
  if any(validCNPU)
     ObservedIDs = unique(...
        AllCells.CNPU_ID(validCNPU));
     FullGrid.Observed = ...
        ismember(...
        FullGrid.CNPU_ID,...
        ObservedIDs);
     %% ---------------------------------------------------
     % Count cells per CNPU
     %% ---------------------------------------------------
     [uniqueCNPU,~,idx] = ...
        unique(...
```

```
        AllCells.CNPU_ID(validCNPU));
    cellCounts = ...
        accumarray(...
        idx,...
        1);
    [isObserved,loc] = ...
        ismember(...
        FullGrid.CNPU_ID,...
        uniqueCNPU);
    FullGrid.NCells(isObserved) = ...
        cellCounts(loc(isObserved));
  end
end
%% ======================================================================
% 6. LOAD BIOLOGICAL TEMPLATE BOUNDARIES
%% ======================================================================
% Biological Stage 13 uses the Stage 14 Sox2 template.
Boundary14 = ...
  getSox2Boundary("Stage13",cfg);
% Biological Stage 14 uses the Stage 15 Sox2 template.
Boundary15 = ...
  getSox2Boundary("Stage14",cfg);
%% ======================================================================
% 7. BIOLOGICAL TEMPLATE MEMBERSHIP
%% ======================================================================
FullGrid.Stage13Template = ...
  pointsInsideBoundary(...
  FullGrid.CNPU_CenterX,...
  FullGrid.CNPU_CenterY,...
  Boundary14);
FullGrid.Stage14Template = ...
  pointsInsideBoundary(...
  FullGrid.CNPU_CenterX,...
  FullGrid.CNPU_CenterY,...
  Boundary15);
FullGrid.AnySox2Template = ...
  FullGrid.Stage13Template | ...
  FullGrid.Stage14Template;
%% ======================================================================
% 8. QC AGAINST EXPECTED LEGACY EXTENT
%% ======================================================================
fprintf('Legacy CNPU size: %.1f x %.1f pixels\n',...
  cfg.CNPUsize,...
  cfg.CNPUsize);
```

```
fprintf('Grid origin: (%.1f, %.1f)\n',...
  cfg.GridOriginX,...
  cfg.GridOriginY);
fprintf('Configured X extent: %.1f to %.1f\n',...
  cfg.ExpectedXRange(1),...
  cfg.ExpectedXRange(2));
fprintf('Configured Y extent: %.1f to %.1f\n',...
  cfg.ExpectedYRange(1),...
  cfg.ExpectedYRange(2));
fprintf('CNPU X index range: %d to %d\n',...
  minCNPU_X,...
  maxCNPU_X);
fprintf('CNPU Y index range: %d to %d\n',...
  minCNPU_Y,...
  maxCNPU_Y);
fprintf('Total complete CNPUs: %d\n',...
  height(FullGrid));
fprintf('Observed CNPUs: %d\n',...
  sum(FullGrid.Observed));
fprintf('Unobserved CNPUs: %d\n',...
  sum(~FullGrid.Observed));
fprintf('CNPUs inside Stage 13 template: %d\n',...
  sum(FullGrid.Stage13Template));
fprintf('CNPUs inside Stage 14 template: %d\n',...
  sum(FullGrid.Stage14Template));
%% ================================================================
% 9. CHECK CELL ASSIGNMENTS AGAINST COMPLETE GRID
%% ================================================================
if ~isempty(AllCells)
  validCNPU = ...
    ismember(...
    AllCells.CNPU_ID,...
    FullGrid.CNPU_ID);
  nOutsideGrid = ...
    sum(~validCNPU);
  fprintf(...
    'Cells assigned outside complete legacy grid: %d\n',...
    nOutsideGrid);
  if nOutsideGrid > 0
    warning([...
      '%d cells were assigned to CNPUs outside the ',...
      'configured legacy grid.'],...
      nOutsideGrid);
  end
```

```
end
end
%% ======================================================================
% LOCAL HELPER
%% ======================================================================
function inside = ...
  pointsInsideBoundary(X,Y,Boundary)
validBoundary = ...
  isfinite(Boundary.X) & ...
  isfinite(Boundary.Y);
Xb = Boundary.X(validBoundary);
Yb = Boundary.Y(validBoundary);
inside = false(size(X));
if numel(Xb) >= 3
  inside = inpolygon(...
    X,...
    Y,...
    Xb,...
    Yb);
end
end
```

Function 6: plotCompleteGridQC

```
function plotCompleteGridQC(FullGrid,cfg)
%% ======================================================================
% COMPLETE CNPU GRID QC
% Visualizes:
%   1. complete CNPU lattice
%   2. observed vs unobserved CNPUs
%   3. Stage 14 Sox2 template
%   4. Stage 15 Sox2 template
%% ======================================================================
fprintf('\n============================================================\n');
fprintf('COMPLETE CNPU GRID QC\n');
fprintf('============================================================\n');
%% ----------------------------------------------------------------
% Load boundaries
%% ----------------------------------------------------------------
Boundary14 = ...
  getSox2Boundary("Stage13",cfg);
Boundary15 = ...
  getSox2Boundary("Stage14",cfg);
%% ----------------------------------------------------------------
```

```
% Figure
%% ---------------------------------------------------------
fig = figure(...
  'Position',cfg.FigurePosition,...
  'Color','w');
hold on;
%% ---------------------------------------------------------
% Unobserved CNPUs
%% ---------------------------------------------------------
unobserved = ...
  ~FullGrid.Observed;
scatter(...
  FullGrid.CNPU_CenterX(unobserved),...
  FullGrid.CNPU_CenterY(unobserved),...
  8,...
  'filled');
%% ---------------------------------------------------------
% Observed CNPUs
%% ---------------------------------------------------------
observed = ...
  FullGrid.Observed;
scatter(...
  FullGrid.CNPU_CenterX(observed),...
  FullGrid.CNPU_CenterY(observed),...
  15,...
  'filled');
%% ---------------------------------------------------------
% Stage 14 template boundary
%% ---------------------------------------------------------
plot(...
  Boundary14.X,...
  Boundary14.Y,...
  'LineStyle',cfg.BoundaryLineStyle,...
  'LineWidth',cfg.BoundaryLineWidth);
%% ---------------------------------------------------------
% Stage 15 template boundary
%% ---------------------------------------------------------
plot(...
  Boundary15.X,...
  Boundary15.Y,...
  'LineStyle','--',...
  'LineWidth',cfg.BoundaryLineWidth);
%% ---------------------------------------------------------
% Axes
```

```
%% ------------------------------------------------------------
axis equal;
xlim([...
   min(FullGrid.CNPU_StartX) ...
   max(FullGrid.CNPU_EndX)]);
ylim([...
   min(FullGrid.CNPU_StartY) ...
   max(FullGrid.CNPU_EndY)]);
set(gca,'YDir','reverse');
xlabel('SuperEmbryo X');
ylabel('SuperEmbryo Y');
title('Complete 9 \times 9 CNPU Grid QC');
box on;
%% ------------------------------------------------------------
% Save
%% ------------------------------------------------------------
outputFile = ...
   fullfile(...
   cfg.OutputRoot,...
   'CompleteCNPUGrid_QC.png');
exportgraphics(...
   fig,...
   outputFile,...
   'Resolution',cfg.FigureResolution);
fprintf(...
   'Total CNPUs: %d\n',...
   height(FullGrid));
fprintf(...
   'Observed CNPUs: %d\n',...
   sum(FullGrid.Observed));
fprintf(...
   'Unobserved CNPUs: %d\n',...
   sum(~FullGrid.Observed));
fprintf(...
   'Saved: %s\n',...
   outputFile);
close(fig);
end
```

Function 7: aggregateGroupCNPU

```
function Results = ...
   aggregateGroupCNPU(...
   AllCells,...
```

```
    FullGrid,...
    EmbryoInfo,...
    cfg)
%% ================================================================
% AGGREGATE EMBRYO-BALANCED GROUP CNPU STATISTICS
% Level 1:
%   cells → embryo × CNPU
% Level 2:
%   embryo × CNPU → group × CNPU
%
% The complete grid is retained, so absent CNPUs are represented explicitly as NaN rather
than disappearing.
%% ================================================================
fprintf('\n============================================================\n');
fprintf('GROUP-LEVEL CNPU AGGREGATION\n');
fprintf('============================================================\n');
%% ------------------------------------------------------------
% Determine analysis scope
%% ------------------------------------------------------------
if cfg.TestMode
    GroupsToAnalyze = string(cfg.TestGroup);
    FeaturesToAnalyze = string(cfg.TestFeature);
else
    GroupsToAnalyze = string(cfg.AllowedGroups);
    FeaturesToAnalyze = string(cfg.Features);
end
%% ------------------------------------------------------------
% Initialize results
%% ------------------------------------------------------------
Results = struct();
%% ------------------------------------------------------------
% Loop over features
%% ------------------------------------------------------------
for f = 1:numel(FeaturesToAnalyze)
    FeatureName = FeaturesToAnalyze(f);
    fprintf('\n============================================================\n');
    fprintf('FEATURE: %s\n',FeatureName);
    fprintf('============================================================\n');
    if ~ismember(...
            FeatureName,...
            AllCells.Properties.VariableNames)
        warning(...
            'Feature %s not found in AllCells.',...
            FeatureName);
```

```
    continue;
end
%% ------------------------------------------------------
% Loop over groups
%% ------------------------------------------------------
for g = 1:numel(GroupsToAnalyze)
    GroupName = GroupsToAnalyze(g);
    fprintf('\n----------------------------------------\n');
    fprintf('GROUP: %s\n',GroupName);
    fprintf('----------------------------------------\n');
    %% ------------------------------------------------
    % Select group
    %% ------------------------------------------------
    G = AllCells(...
        AllCells.Group == GroupName,:);
    if isempty(G)
        warning(...
            'No cells found for group %s.',...
            GroupName);
        continue;
    end
    %% ------------------------------------------------
    % Valid feature values
    %% ------------------------------------------------
    validFeature = ...
        isfinite(G.(FeatureName));
    G = G(validFeature,:);
    if isempty(G)
        warning(...
            'No valid %s values for %s.',...
            FeatureName,...
            GroupName);
        continue;
    end
    %% ================================================
    % LEVEL 1
    % EMBRYO × CNPU
    %% ================================================
    [groupID,...
     embryo,...
     cnpuX,...
     cnpuY] = findgroups(...
        G.Embryo,...
        G.CNPU_X,...
```

```
    G.CNPU_Y);
EmbryoMedian = splitapply(...
    @(x) median(x,'omitnan'),...
    G.(FeatureName),...
    groupID);
NCells = splitapply(...
    @(x) numel(x),...
    G.(FeatureName),...
    groupID);
EmbryoCNPU = table(...
    embryo,...
    cnpuX,...
    cnpuY,...
    EmbryoMedian,...
    NCells,...
    'VariableNames',{...
    'Embryo',...
    'CNPU_X',...
    'CNPU_Y',...
    'EmbryoMedian',...
    'NCells'});
%% Spatial coordinates
EmbryoCNPU.CNPU_CenterX = ...
    cfg.GridOriginX + ...
    EmbryoCNPU.CNPU_X .* ...
    cfg.CNPUsize + ...
    cfg.CNPUsize/2;
EmbryoCNPU.CNPU_CenterY = ...
    cfg.GridOriginY + ...
    EmbryoCNPU.CNPU_Y .* ...
    cfg.CNPUsize + ...
    cfg.CNPUsize/2;
EmbryoCNPU.Group = ...
    repmat(...
    GroupName,...
    height(EmbryoCNPU),1);
EmbryoCNPU.Feature = ...
    repmat(...
    FeatureName,...
    height(EmbryoCNPU),1);
%% ========================================================
% LEVEL 2
% GROUP × CNPU
%% ========================================================
```

```
[cnpuGroup,...
 groupCNPU_X,...
 groupCNPU_Y] = findgroups(...
    EmbryoCNPU.CNPU_X,...
    EmbryoCNPU.CNPU_Y);
GroupMedian = splitapply(...
    @(x) median(x,'omitnan'),...
    EmbryoCNPU.EmbryoMedian,...
    cnpuGroup);
GroupSD = splitapply(...
    @(x) std(x,0,'omitnan'),...
    EmbryoCNPU.EmbryoMedian,...
    cnpuGroup);
NEmbryos = splitapply(...
    @(x) sum(isfinite(x)),...
    EmbryoCNPU.EmbryoMedian,...
    cnpuGroup);
TotalCells = splitapply(...
    @(x) sum(x),...
    EmbryoCNPU.NCells,...
    cnpuGroup);
%% ------------------------------------------------------------
% Number of biological embryos in this experimental group
%
% AllCells already contains one Group label for every cell and the
% canonical biological Embryo ID, so group membership is derived
% directly from the loaded cell-level data.
%% ------------------------------------------------------------

groupEmbryos = unique(...
    AllCells.Embryo(...
    AllCells.Group == GroupName));

NGroupEmbryos = ...
    numel(groupEmbryos);

CoveragePct = ...
    100 .* ...
    NEmbryos ./ ...
    NGroupEmbryos;
%% ============================================================
% Build complete-grid group table
%% ============================================================
GroupCNPU = FullGrid(:,{...
```

```
    'CNPU_X',...
    'CNPU_Y',...
    'CNPU_StartX',...
    'CNPU_EndX',...
    'CNPU_StartY',...
    'CNPU_EndY',...
    'CNPU_CenterX',...
    'CNPU_CenterY',...
    'CNPU_ID'});
%% Match observed group statistics onto complete grid
[isPresent,loc] = ismember(...
    [GroupCNPU.CNPU_X GroupCNPU.CNPU_Y],...
    [groupCNPU_X groupCNPU_Y],...
    'rows');
GroupCNPU.GroupMedian = nan(height(GroupCNPU),1);
GroupCNPU.GroupSD = nan(height(GroupCNPU),1);
GroupCNPU.NEmbryos = zeros(height(GroupCNPU),1);
GroupCNPU.CoveragePct = zeros(height(GroupCNPU),1);
GroupCNPU.NCells = zeros(height(GroupCNPU),1);
GroupCNPU.GroupMedian(isPresent) = ...
    GroupMedian(loc(isPresent));
GroupCNPU.GroupSD(isPresent) = ...
    GroupSD(loc(isPresent));
GroupCNPU.NEmbryos(isPresent) = ...
    NEmbryos(loc(isPresent));
GroupCNPU.CoveragePct(isPresent) = ...
    CoveragePct(loc(isPresent));
GroupCNPU.NCells(isPresent) = ...
    TotalCells(loc(isPresent));
GroupCNPU.Group = ...
    repmat(GroupName,height(GroupCNPU),1);
GroupCNPU.Feature = ...
    repmat(FeatureName,height(GroupCNPU),1);
%% ========================================================
% Optional legacy smoothing
%% ========================================================
GroupCNPU.SmoothedMedian = ...
    nan(height(GroupCNPU),1);
GroupCNPU.SmoothedZ = ...
    nan(height(GroupCNPU),1);
if cfg.DoSmoothing
    halfWindow = ...
        floor(cfg.SmoothWindow/2);
    for i = 1:height(GroupCNPU)
```

```
        x0 = GroupCNPU.CNPU_X(i);
        y0 = GroupCNPU.CNPU_Y(i);
        neighborMask = ...
            abs(GroupCNPU.CNPU_X-x0) <= halfWindow & ...
            abs(GroupCNPU.CNPU_Y-y0) <= halfWindow & ...
            isfinite(GroupCNPU.GroupMedian);
        if any(neighborMask)
            GroupCNPU.SmoothedMedian(i) = ...
                mean(...
                GroupCNPU.GroupMedian(neighborMask),...
                'omitnan');
        end
    end
    validSmooth = ...
        isfinite(GroupCNPU.SmoothedMedian);
    if any(validSmooth)
        mu = mean(...
            GroupCNPU.SmoothedMedian(validSmooth));
        sigma = std(...
            GroupCNPU.SmoothedMedian(validSmooth),...
            0);
        if sigma > 0
            GroupCNPU.SmoothedZ(validSmooth) = ...
                (GroupCNPU.SmoothedMedian(validSmooth)-mu) ...
                ./ sigma;
        end
    end
end
%% ========================================================
% QC
%% ========================================================
fprintf('Embryos in group: %d\n',...
    NGroupEmbryos);
fprintf('Embryo/CNPU observations: %d\n',...
    height(EmbryoCNPU));
fprintf('Complete grid CNPUs: %d\n',...
    height(GroupCNPU));
fprintf('CNPUs with data: %d\n',...
    sum(isfinite(GroupCNPU.GroupMedian)));
if any(isfinite(GroupCNPU.CoveragePct))
    fprintf(...
        'Coverage range: %.2f%% to %.2f%%\n',...
        min(GroupCNPU.CoveragePct),...
        max(GroupCNPU.CoveragePct));
```

```
        end
        %% ============================================================
        % Store results
        %% ============================================================
        Results.(FeatureName).(GroupName).EmbryoCNPU = ...
            EmbryoCNPU;
        Results.(FeatureName).(GroupName).GroupCNPU = ...
            GroupCNPU;
        Results.(FeatureName).(GroupName).NGroupEmbryos = ...
            NGroupEmbryos;
    end
end
end
```

Function 8: calculateSharedColorLimits

```
function ColorLimits = ...
    calculateSharedColorLimits(Results,cfg)
%% ====================================================================
% CALCULATE SHARED FIGURE COLOR LIMITS
% One common color scale is calculated for each feature across all groups represented in
Results.
% Uses raw GroupMedian values.
% Smoothed values are NOT used for determining the primary color scale.
%% ====================================================================
fprintf('\n============================================================\n');
fprintf('CALCULATING SHARED COLOR LIMITS\n');
fprintf('============================================================\n');
ColorLimits = struct();
%% ------------------------------------------------------------
% Features actually present
%% ------------------------------------------------------------
FeatureNames = fieldnames(Results);
for f = 1:numel(FeatureNames)
    FeatureName = FeatureNames{f};
    allValues = [];
    GroupNames = fieldnames(...
        Results.(FeatureName));
    for g = 1:numel(GroupNames)
        GroupName = GroupNames{g};
        if ~isfield(...
                Results.(FeatureName).(GroupName),...
                'GroupCNPU')
            continue;
```

```
        end
        T = ...
            Results.(FeatureName).(GroupName).GroupCNPU;
        if ~ismember(...
                'GroupMedian',...
                T.Properties.VariableNames)
            continue;
        end
        values = T.GroupMedian;
        values = values(isfinite(values));
        allValues = ...
            [allValues; values]; %#ok<AGROW>
    end
    %% ---------------------------------------------------------
    % Validate
    %% ---------------------------------------------------------
    if isempty(allValues)
        warning(...
            'No valid values found for %s.',...
            FeatureName);
        ColorLimits.(FeatureName) = [0 1];
        continue;
    end
    %% ---------------------------------------------------------
    % Shared min/max
    %% ---------------------------------------------------------
    lo = min(allValues);
    hi = max(allValues);
    %% Constant feature protection
    if lo == hi
        delta = max(abs(lo)*0.05,1e-6);
        lo = lo - delta;
        hi = hi + delta;
    end
    ColorLimits.(FeatureName) = ...
        [lo hi];
    fprintf(...
        '%-25s  [%g, %g]\n',...
        FeatureName,...
        lo,...
        hi);
end
end
```

Function 9: plotGroupSpatialMaps

```
function plotGroupSpatialMaps(...
  Results,...
  FullGrid,...
  ColorLimits,...
  cfg)
%% ================================================================
% PLOT GROUP SPATIAL MAPS
% For each spectral feature, generates THREE figures:
%   1. Population central tendency
%      = embryo-balanced median feature value
%   2. Population variability
%      = standard deviation across embryo-level CNPU medians
%   3. Spatial coverage
%      = (# embryos contributing / total embryos in group) * 100
% Each figure contains one panel per experimental group.
%
% IMPORTANT:
%   The Sox2 boundary is an anatomical OVERLAY ONLY.
%   It does not mask the data or define the plotted region.
% The complete legacy CNPU grid defines the common spatial canvas.
%% ================================================================
fprintf('GENERATING PRIMARY SPATIAL MAPS\n');
fprintf('=============================================================\n');
%% ================================================================
% 1. FEATURES
%% ================================================================
FeatureNames = ...
  fieldnames(Results);
%% ================================================================
% 2. PRELOAD COMMON BOUNDARIES
% Stage 13 groups use Stage 14 template.
% Stage 14 groups use Stage 15 template.
%% ================================================================
BoundaryStage13 = ...
  getSox2Boundary(...
  "Stage13",...
  cfg);
BoundaryStage14 = ...
  getSox2Boundary(...
  "Stage14",...
  cfg);
%% ================================================================
% 3. LOOP OVER FEATURES
```

```
%% ============================================================
for f = 1:numel(FeatureNames)
  FeatureName = ...
    FeatureNames{f};
  GroupNames = ...
    fieldnames(...
    Results.(FeatureName));
  if isempty(GroupNames)
    continue;
  end
  %% ============================================================
  % FIGURE 1
  % POPULATION CENTRAL TENDENCY
  %% ============================================================
  createMetricFigure(...
    Results,...
    FullGrid,...
    FeatureName,...
    GroupNames,...
    'GroupMedian',...
    'Population Central Tendency',...
    ColorLimits.(FeatureName),...
    BoundaryStage13,...
    BoundaryStage14,...
    cfg,...
    'PopulationMedian');
  %% ============================================================
  % FIGURE 2
  % POPULATION VARIABILITY
  %% ============================================================
  % Calculate shared SD color limits across groups
  %% ------------------------------------------------------------
  sdValues = [];
  for g = 1:numel(GroupNames)
    GroupName = ...
      GroupNames{g};
    T = ...
      Results.(FeatureName).(GroupName).GroupCNPU;
    validSD = ...
      isfinite(T.GroupSD);
    sdValues = ...
      [sdValues; ...
       T.GroupSD(validSD)]; %#ok<AGROW>
  end
```

```
    if isempty(sdValues)
        variabilityLimits = [0 1];
    else
        variabilityLimits = [...
            min(sdValues),...
            max(sdValues)];
        if variabilityLimits(1) == ...
                variabilityLimits(2)
            delta = ...
                max(abs(variabilityLimits(1))*0.05,...
                1e-6);
            variabilityLimits = ...
                variabilityLimits + [-delta delta];
        end
    end
    createMetricFigure(...
        Results,...
        FullGrid,...
        FeatureName,...
        GroupNames,...
        'GroupSD',...
        'Population Variability (SD)',...
        variabilityLimits,...
        BoundaryStage13,...
        BoundaryStage14,...
        cfg,...
        'PopulationSD');
    %% ============================================================
    % FIGURE 3
    % SPATIAL COVERAGE
    %% ============================================================
    createMetricFigure(...
        Results,...
        FullGrid,...
        FeatureName,...
        GroupNames,...
        'CoveragePct',...
        'Spatial Coverage',...
        [0 100],...
        BoundaryStage13,...
        BoundaryStage14,...
        cfg,...
        'SpatialCoverage');
end
```

```
end
%% ======================================================================
% LOCAL FUNCTION
% CREATE ONE FOUR-PANEL METRIC FIGURE
%% ======================================================================
function createMetricFigure(...
   Results,...
   FullGrid,...
   FeatureName,...
   GroupNames,...
   MetricName,...
   MetricTitle,...
   colorLimits,...
   BoundaryStage13,...
   BoundaryStage14,...
   cfg,...
   OutputLabel)
%% ----------------------------------------------------------------
% Create figure
%% ----------------------------------------------------------------
fig = figure(...
   'Position',cfg.FigurePosition,...
   'Color','w');
nGroups = ...
   numel(GroupNames);
nCols = 2;
nRows = ceil(nGroups/nCols);
tiledlayout(...
   nRows,...
   nCols,...
   'TileSpacing','compact',...
   'Padding','compact');
%% ======================================================================
% Plot each experimental group
%% ======================================================================
for g = 1:nGroups
   GroupName = ...
      GroupNames{g};
   nexttile;
   T = ...
      Results.(FeatureName).(GroupName).GroupCNPU;
   %% -------------------------------------------------------
   % Metric
   %% -------------------------------------------------------
```

```
values = ...
    T.(MetricName);
valid = ...
    isfinite(values);
%% ------------------------------------------------------
% Plot all valid CNPUs
% NO Sox2 masking.
%% ------------------------------------------------------
hold on;
if any(valid)
    scatter(...
        T.CNPU_CenterX(valid),...
        T.CNPU_CenterY(valid),...
        30,...
        values(valid),...
        'filled');
end
%% ------------------------------------------------------
% Appropriate Sox2 boundary
%% ------------------------------------------------------
if startsWith(...
        string(GroupName),...
        "Stage13")
    Boundary = ...
        BoundaryStage13;
elseif startsWith(...
        string(GroupName),...
        "Stage14")
    Boundary = ...
        BoundaryStage14;
else
    error(...
        'Unrecognized group name: %s',...
        GroupName);
end
%% ------------------------------------------------------
% Overlay boundary only
%% ------------------------------------------------------
plot(...
    Boundary.X,...
    Boundary.Y,...
    'LineStyle',...
    cfg.BoundaryLineStyle,...
    'LineWidth',...
```

```
    cfg.BoundaryLineWidth);
%% ------------------------------------------------------
% Shared color scale
%% ------------------------------------------------------
clim(colorLimits);
%% ------------------------------------------------------
% Complete legacy spatial canvas
%% ------------------------------------------------------
xMin = ...
    min(FullGrid.CNPU_StartX);
xMax = ...
    max(FullGrid.CNPU_EndX);
yMin = ...
    min(FullGrid.CNPU_StartY);
yMax = ...
    max(FullGrid.CNPU_EndY);
xlim([xMin xMax]);
ylim([yMin yMax]);
axis equal;
%% ------------------------------------------------------
% Image-style orientation
%% ------------------------------------------------------
set(gca,'YDir','reverse');
xlabel('SuperEmbryo X');
ylabel('SuperEmbryo Y');
title(...
    strrep(GroupName,'_','\_'),...
    'Interpreter','tex');
box on;
%% ------------------------------------------------------
% QC annotation
%% ------------------------------------------------------
nValid = ...
    sum(valid);
nTotal = ...
    height(T);
if strcmp(MetricName,'GroupMedian')
    annotationText = ...
        sprintf(...
        'Valid CNPUs: %d / %d',...
        nValid,...
        nTotal);
elseif strcmp(MetricName,'GroupSD')
    annotationText = ...
```

```
            sprintf(...
            'Valid CNPUs: %d / %d',...
            nValid,...
            nTotal);
    elseif strcmp(MetricName,'CoveragePct')
        maxCoverage = ...
            max(values,[],'omitnan');
        annotationText = ...
            sprintf(...
            'Max coverage: %.1f%%',...
            maxCoverage);
    else
        annotationText = '';
    end
    text(...
        0.02,...
        0.98,...
        annotationText,...
        'Units','normalized',...
        'VerticalAlignment','top',...
        'FontSize',8);
    hold off;
end
%% ======================================================================
% Shared colorbar
%% ======================================================================
cb = colorbar;
cb.Layout.Tile = 'east';
switch MetricName
    case 'GroupMedian'
        cb.Label.String = ...
            sprintf('%s — Median',FeatureName);
    case 'GroupSD'
        cb.Label.String = ...
            sprintf('%s — SD',FeatureName);
    case 'CoveragePct'
        cb.Label.String = ...
            'Embryo coverage (%)';
end
%% ======================================================================
% Figure title
%% ======================================================================
sgtitle(...
    sprintf(...
```

```
      '%s — %s',...
      FeatureName,...
      MetricTitle),...
  'Interpreter','none');
%% ================================================================
% Save
%% ================================================================
safeFeatureName = ...
  matlab.lang.makeValidName(FeatureName);
outputFile = ...
  fullfile(...
  cfg.OutputRoot,...
  sprintf(...
      '%s_%s_GroupSpatialMaps.png',...
      safeFeatureName,...
      OutputLabel));
exportgraphics(...
  fig,...
  outputFile,...
  'Resolution',...
  cfg.FigureResolution);
fprintf(...
  'Saved: %s\n',...
  outputFile);
close(fig);
end
```

Function 10 (technically called in Function 9, not the driver): getSox2Boundary

```
function Boundary = ...
  getSox2Boundary(GroupName,cfg)
%% ================================================================
% GET SOX2 TEMPLATE BOUNDARY
%
% Biological Stage 13:
%   uses Stage 14 Sox2 template
%
% Biological Stage 14:
%   uses Stage 15 Sox2 template
%
% Output:
%   Boundary.X
%   Boundary.Y
%   Boundary.Group
```

```
%   Boundary.File
%   Boundary.XRange
%   Boundary.YRange
%% ======================================================================
GroupName = string(GroupName);
%% ======================================================================
% 1. DETERMINE BIOLOGICAL STAGE
%% ======================================================================
if startsWith(GroupName,"Stage13")
   boundaryFile = ...
      cfg.Sox2BoundaryFiles.Stage13;
elseif startsWith(GroupName,"Stage14")
   boundaryFile = ...
      cfg.Sox2BoundaryFiles.Stage14;
else
   error(...
      'Unrecognized group name: %s',...
      GroupName);
end
%% ======================================================================
% 2. CHECK FILE
%% ======================================================================
if ~isfile(boundaryFile)
   error(...
      'Sox2 boundary file not found:\n%s',...
      boundaryFile);
end
%% ======================================================================
% 3. READ BOUNDARY FILE
%% ======================================================================
T = readtable(...
   boundaryFile,...
   'VariableNamingRule','preserve');
vars = T.Properties.VariableNames;
fprintf('\nSox2 boundary: %s\n',...
   boundaryFile);
fprintf('Boundary columns: %s\n',...
   strjoin(vars,', '));
%% ======================================================================
% 4. IDENTIFY X/Y COLUMNS
%% ======================================================================
xName = '';
yName = '';
%% ----------------------------------------------------------------
```

```
% Preferred explicit names
%% -------------------------------------------------------------
xCandidates = {...
   'X',...
   'V1',...
   'col',...
   'Centroid_1',...
   'Boundary_1',...
   'Var1'};
yCandidates = {...
   'Y',...
   'V2',...
   'row',...
   'Centroid_2',...
   'Boundary_2',...
   'Var2'};
for k = 1:numel(xCandidates)
   idx = find(...
      strcmpi(vars,xCandidates{k}),...
      1);
   if ~isempty(idx)
      xName = vars{idx};
      break;
   end
end
for k = 1:numel(yCandidates)
   idx = find(...
      strcmpi(vars,yCandidates{k}),...
      1);
   if ~isempty(idx)
      yName = vars{idx};
      break;
   end
end
%% -------------------------------------------------------------
% Generic two-column fallback
%
% Some boundary CSVs are imported by MATLAB as Var1/Var2.
% If exactly two columns exist, interpret them as X/Y.
%% -------------------------------------------------------------
if (isempty(xName) || isempty(yName)) && ...
      width(T) == 2
   xName = vars{1};
   yName = vars{2};
```

```
    fprintf(...
        'Using generic two-column boundary format: %s, %s\n',...
        xName,...
        yName);
end
%% ====================================================================
% 5. VALIDATE COLUMN IDENTIFICATION
%% ====================================================================
if isempty(xName) || isempty(yName)
    error([...
        'Could not identify X/Y columns in Sox2 boundary ',...
        'file:\n%s\n\n',...
        'Available columns:\n%s'],...
        boundaryFile,...
        strjoin(vars,', '));
end
%% ====================================================================
% 6. EXTRACT NUMERIC COORDINATES
%% ====================================================================
try
    X = double(T.(xName));
    Y = double(T.(yName));
catch ME
    error([...
        'Sox2 boundary coordinates could not be converted ',...
        'to numeric values.\n',...
        'File: %s\n',...
        'X column: %s\n',...
        'Y column: %s\n',...
        'Original error: %s'],...
        boundaryFile,...
        xName,...
        yName,...
        ME.message);
end
%% ====================================================================
% 7. REMOVE INVALID ROWS
%% ====================================================================
valid = ...
    isfinite(X) & ...
    isfinite(Y);
X = X(valid);
Y = Y(valid);
if numel(X) < 3
```

```
    error(...
        'Sox2 boundary contains fewer than 3 valid points:\n%s',...
        boundaryFile);
end
%% ====================================================================
% 8. CLOSE POLYGON
%% ====================================================================
if X(1) ~= X(end) || ...
   Y(1) ~= Y(end)
    X(end+1,1) = X(1);
    Y(end+1,1) = Y(1);
end
%% ====================================================================
% 9. RETURN STANDARDIZED STRUCTURE
%% ====================================================================
Boundary = struct();
Boundary.X = X;
Boundary.Y = Y;
Boundary.Group = ...
    GroupName;
Boundary.File = ...
    boundaryFile;
Boundary.XRange = ...
    [min(X) max(X)];
Boundary.YRange = ...
    [min(Y) max(Y)];
%% ====================================================================
% 10. QC
%% ====================================================================
fprintf(...
    'Valid boundary points: %d\n',...
    numel(X));
fprintf(...
    'Boundary X range: %.6f to %.6f\n',...
    min(X),...
    max(X));
fprintf(...
    'Boundary Y range: %.6f to %.6f\n',...
    min(Y),...
    max(Y));
end
```